

Handheld Device Programming II - Android

Introduction

What is Android?

Android is an **open-source mobile operating system** primarily designed for smartphones and tablets, but it is also widely used in smart TVs, smart watches, and car systems. Android was originally developed by Android Inc. and later acquired and maintained by Google. Android is built on top of the **Linux Kernel**, which makes it secure, stable, and highly efficient.

Devices that use Android

Android is not limited to mobile phones only. It is used across a wide range of devices:

- Smartphones
- Tablets
- Smart TVs (Android TV)
- Smart Watches (Wear OS)
- Car infotainment systems (Android Auto)

Key Features of Android

Android has become one of the most popular operating systems due to its powerful features:

- **Open Source** - Developers can modify and customize the system freely
- **User-Friendly Interface** - Simple, flexible, and intuitive UI
- **Multitasking Support** - Multiple applications can run simultaneously
- **Hardware Integration** - Easy access to camera, GPS, Bluetooth, sensors, etc.
- **Large App Ecosystem** - Millions of apps available on Google Play Store

Tools Used for Android Development

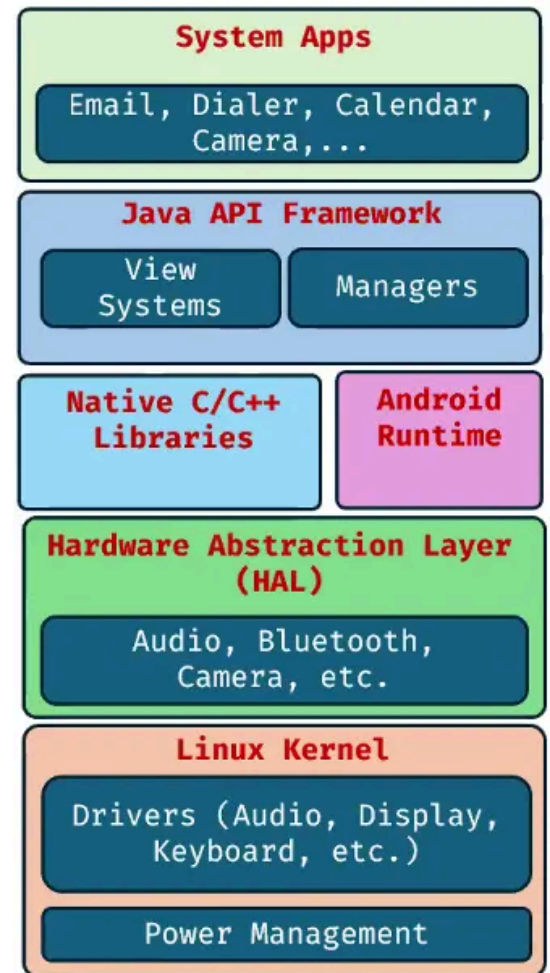
To develop Android applications, the following tools are commonly used:

- **Android Studio** - Official Integrated Development Environment (IDE)
- **Java / Kotlin** - Programming languages for app development
- **XML** - Used for UI design
- **Android SDK** - Provides APIs, libraries, and development tools
- **Emulator or Real Device** - Used for testing applications

Android Architecture

Android follows a layered architecture:

1. **Linux Kernel** - Manages hardware, memory, processes, and device drivers
2. **Native Libraries** - Provide support for media, graphics, and databases
3. **Android Runtime (ART)** - Executes Android applications
4. **Application Framework** - Provides core services such as Activities, Services, Notifications
5. **Applications** - User-installed and system apps (e.g: YouTube, WhatsApp)



Platform architecture

Android Versions

Android versions were initially named after **desserts**, following alphabetical order. Later versions use numeric naming. Each Android version is also associated with an **SDK (API) Level**, which is important for app compatibility and development.

[Android API Levels](#)

Setting Up Android Studio

Android Studio is the official IDE for Android development.

Steps:

- [Install Android Studio](#)
- Install required SDK components
- Configure emulator or connect a real device

Android Project Structure

Android Package Structure

Android package structure එක හඳුනාකොට ඉතාමත් වැදගත් point එකක් නමයි package name එක (එකට Application ID කියලා කියන්නත් පුළුවන්). Package name එක කියලා කියන්නේ unique string එකක්. මේ unique string එක නිසා අපි හඳුනා application එක ඔයාගේ Android phone එකේ වගේම Google Play Store එකෙහිත් වෙන් කරලා හඳුනාගනී. මේක හරියට ඔයා හඳුනා application එකේ digital fingerprint එක වගේ.

- Package Name (Application ID)
- This is unique string
- that identifies your app on your android device and Google Play Store
- this acts as a digital fingerprint for your application

Naming Convention (Reverse Domain Name)

Package name එක හඳුනාකොට standard practice එක නමයි reverse domain name system එක පාවිච්චි කරන එක. මේකෙන් අපි තහවුරු කරගන්නවා අපි දාන නම globally unique කරගන්නවා කියලා.

 suffix.domain.app_name

Ex: lk.jiat.myapplication

lk: Top-level country domain (in this case - Sri Lanka)

jiat: Organization or Company Name

myapplication: Specific name of my application

Why is this important?

- Play Store uniqueness - එකම package name එකෙන් Play Store එකේ app දෙකක් තියෙන්න බෑ. උදාහරණයක් විදිහට, com.example.myapplication කියන package name එකෙන් කවුරුහරි කෙනෙක් app එකක් දාලා තියෙනවා නම් ඔයාට මේ package name එක නැවත පාවිච්චි කරන්න බෑ (upload කරන්න බෑ).
 - Play Store URL = <https://play.google.com/store/apps/details?id=lk.jiat.myapplication>
 - අපේ app එක සඳහා මේ package name එක පාවිච්චි කරලා unique URL එකක් හදලා දෙනවා.
- Device identification - Android විසින් මේ package name එක පාවිච්චි කරනවා අපේ phone එකේ install කරලා තියෙන අනෙක් app වලින් වෙන් කරලා තියාගන්න.
 - Ex: lk.jiat.application
 - අපි එකම package name එකෙන් app දෙකක් install කරන්න ගියොත් දෙවෙනි app එකෙන් පලවෙනි app එක override කරනවා.
- අපේ package name එකෙන් project එකක් හදන්න කියල කිව්වම Android Studio IDE එකෙන් automatically nested folders වගයක් හදල දෙනවා.

Project Folder → When use lk.jiat.myapplication → IDE

Folder Path: app/src/main/java/lk/jiat/**myapplication**

- myapplication
 - MainActivity.kt
 - MainActivity.java
 - adapter/
 - config/
 - util/

Important Rules

- Lowercase only

- Stay Consistent - එක පාරක් ඔයා Play Store එකට app එක දාලා publish කළාට පස්සේ නැවත වෙනස් කරන්න බෑ. එහෙම වෙනස් කලොත් එක වෙනම app එකක් විදිහට තමා identify වෙන්නේ.
- Avoid "example" as a package name - professional project හදනකොට මේ වගේ වචන පාවිච්චි කරන එක සුදුසු නෑ.

Summary

Part	Description	Example
Top-Level Domain	Represents the country or type of org	com, org, lk
Company Name	Brand Name	jitat, google, facebook
App Name	Specific name for the application	notepad, calculator, calendar Ex: • lk.jiat.notepad • lk.jiat.calculator • lk.jiat.calendar

What is an Android Project?

- Android project එකක් කියල කියන්නේ source code, resources හා configuration files වල structured එකතුවක් (අපි අලුතින් ප්රොජෙට්ටි එකක් හදනකොට IDE එකෙන් automatically මේ ටික හදල දෙනවා)

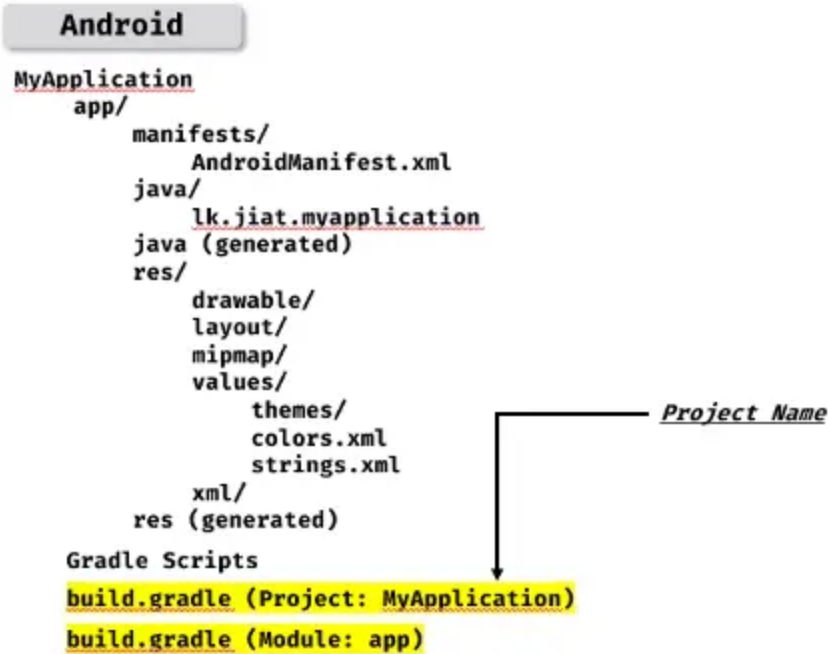
Android Studio Project Views

- Android Studio එකෙන් අපිට project එකක් view කරන්න ක්‍රම කිහිපයක් හදල දෙනවා.

View	Description
Android (Default)	Simplified and logical view (Familiar for developers)
Project	Actual Gradle-based directory structure

Packages	Java/Kotlin package-based view
----------	--------------------------------

Top-Level Project Structure



app/ Folder Structure

අපේ application එකේ main module එක නමයි මේක. මේකේ ඇතුලේ application එකට අවශ්‍ය source code සහ resources තියෙනවා.

- src/
- build.gradle (Module-level gradle)

src/ Folder Structure

- androidTest/
- main/
- test/

Folder	Purpose
main/	Actual application code

test/	Local unit test (JVM)
androidTest/	Instrumented tests (Physical device or emulator)

main/ Folder Structure

- AndroidManifest.xml
- java/
- res/

AndroidManifest.xml

- මේ file එක නමයි අපේ application එකේ core configuration file එක

Responsibilities

- Application package name
- Activities
- Permissions (storage/location/camera,...)
- Services
- Broadcast Receivers
- App theme and icon
- Launcher Activity

```

<?xml version="1.0" encoding="utf-8"?>
<manifest>

    <application
        ...
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApplication">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>

```

java/ Folder Structure

මේ folder එක ඇතුළේ අපි ලියන සියලුම application logic ටික තියාගනී.

- lk/jiat/myapplication/
 - MainActivity.java
 - LoginActivity.java
 - util/
 - adapters/
 - model/

Common Packages

Package	Description
---------	-------------

activity/	UI screens
adapter/	RecyclerView Adapters
model/	Data models
util/	Helper/ Utility class
viewmodel/	MVVM View Models (Model-View-View-Model)

res/ Folder Structure (Resources)

මේ folder එක ඇතුලේ තමයි non-code resources තියෙන්නේ.

- res/
 - layout/
 - drawable/
 - mipmap/
 - values/
 - menu/
 - raw/

Resource Folder	Description
layout/	Contains all XML layout files for UI
drawable/	Used for images or shape • PNG/JPG images • Vector graphics • Shape XML files
values/	Store reusable values • colors.xml • strings.xml

	• styles.xml • themes.xml • dimens.xml
menu/	Defines toolbar and navigation menus
raw/	Store raw files • Audio • Video • JSON files

Gradle Files

Project-Level Gradle file - build.gradle (Project: **MyApplication**)

- Global configuration
- Plugin versions
- Repositories (Google, Maven central)

Module-Level Gradle file - build.gradle (Module: **app**)

- SDK versions
- Dependencies
- Build types

```

android {
    namespace 'lk.codehive.myapplication'
    compileSdk {
        version = release(36)
    }

    defaultConfig {
        applicationId "lk.codehive.myapplication"
        minSdk 28
        targetSdk 36
        versionCode 1
        versionName "1.0"
    }
    ...
}

dependencies {
    implementation 'android.appcompat.appcompat:1.6.1'
    .....
}

```

compileSDK, targetSDK and minSDK

මෙන්න මේ තුන්දෙනා යටතේ අපි කියනවා:

- මොන Android version වලද run කරන්න පුළුවන්
- මොන Android version එකටද support කරන්නේ
- මොකක් මතද run කරන්න පුළුවන්
- These three values define **which android version our app can use, support and run on**

මේ ගැන අපි කියන්න ඕනේ **module level gradle file එකේ**.

compileSdkVerion (compileSdk)

- මේකෙන් define කරන්නේ අපේ app එක compile කරන්න පාවිච්චි කරන Android API level එක ගැන
- මේකෙන් තීරණය කරන්න පුළුවන් මොන වගේ features සහ API අපිට පාවිච්චි කරන්න පුළුවන්ද කියන එක
- මේක අපිට වලංගු වෙන්නේ **development අවස්ථාවේ පමණයි, Runtime එකේදී නෙවෙයි.**

Ex: compileSdk 34 - Android version 14 දක්වා APIs පාවිච්චි කරන්න පුළුවන්. මේකෙන් අදහස් කරන්නේ නෑ Android 14 තියෙන phone එකක විතරක් run වෙන එක

minSdkVersion (minSdk)

- අපේ app එකට ඕන කරන අවම Android version එක නමා මෙනහ define කරන්නේ.
- මේ version එකෙන් පහළ තියෙන phone වල මේ app එක run කරන්න බෑ.

Ex: minSdk 28 - Android 9 හා ඊට උඩින් තියෙන version එකක app එක run කරන්න පුළුවන්

targetSdkVersion (targetSdk)

- අපේ app වල design කරන්නේ හා test කරන්න පාවිච්චි කරන version එක
- මේක වලංගු වන්නේ runtime එකේදී පමණි

Ex: targetSdk 34 - App follows Android 14 (UPSIDE_DONW_CAKE) behavior rules. Required for Play Store submission

Key Points:

- Device version → targetSdk → system applies **backward compatibility**
- If targetSdk is low → system applies **legacy behavior**

Summary

Property	Purpose	Affects Installation	Affects Runtime
compileSdk	APIs can use	No	No
minSdk	Minimum OS required	Yes	Yes
targetSdk	OS behavior app target	No	Yes

Basic UI Designing

Android වලදී UI හඳුනාකොට Java හරි Kotlin වලින් හරි designing code ලියනවා වෙනුවට XML (eXtensible Markup Language) නමයි පාවිච්චි කරලා තියෙන්නේ.

- UI වලට අදාළ කොටසයි, application logic එකයි separate කරලා තියෙන එකෙන්,
 - Code reusability
 - Maintainability
 - Code readability
 - Team collaboration (Developers and Designers)

Most UI components are defined using **XML(eXtensible Markup Language)** instead of writing UI code in **Java or Kotlin**

this separation of UI design and application logic that improves:

- **code reusability**
- **Maintainability**
- **Code readability**
- **Team collaboration(Developers and Designers)**

Why XML is used in Android

Android වලදී XML පාවිච්චි කරලා වැඩ කිහිපයක් කරනවා,

By using XML in android we can do several things,

- **Designing UI**
- **Store values(colors.xml, strings.xml, dimens.xml)**
- **Configure applications components**

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    ...
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".activity.LoginActivity">

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Button"
        .../>

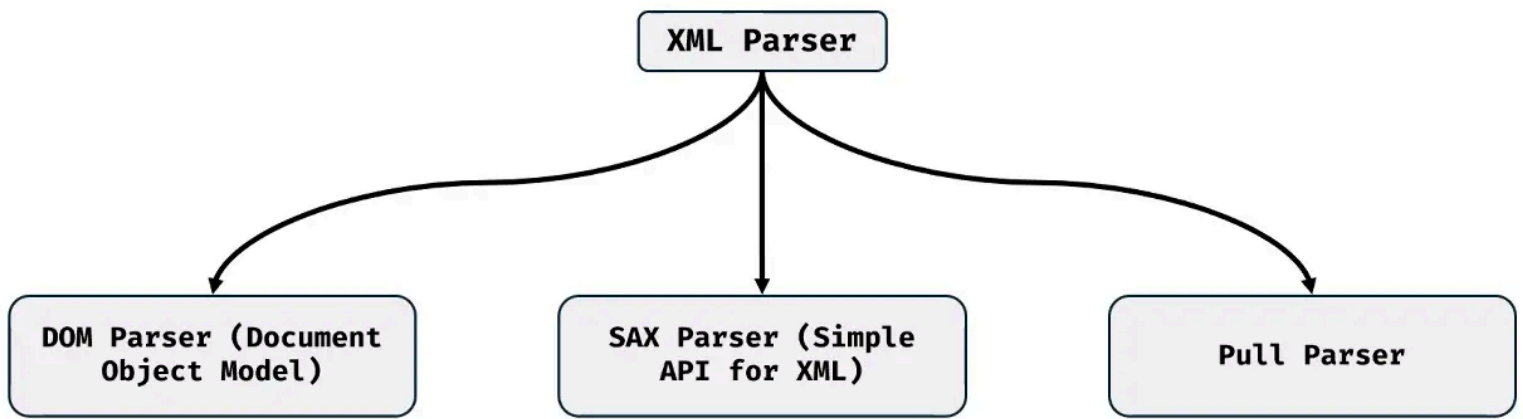
</androidx.constraintlayout.widget.ConstraintLayout>
```

Key components of XML Layout

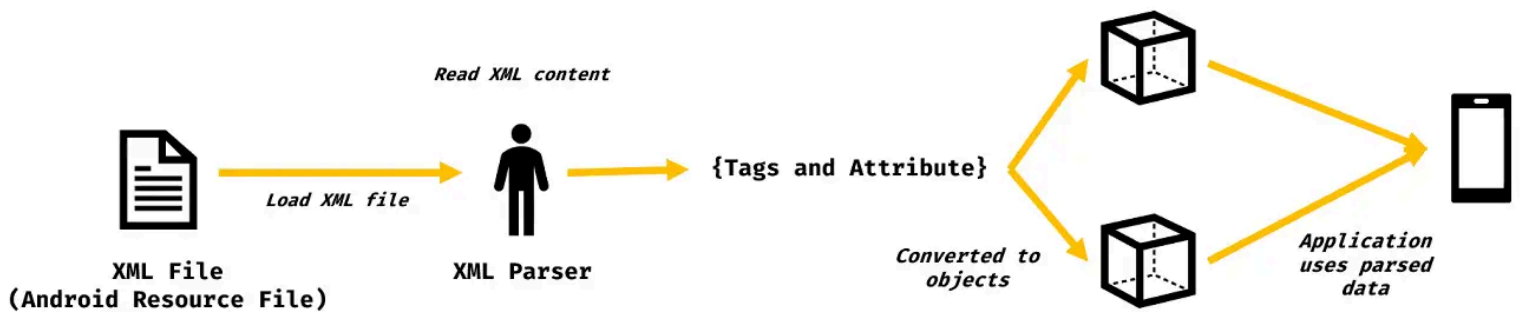
Component	Description
Root Layout	Parent Container (ConstraintLayout, RelativeLayout, LinearLayout)
View	UI element (Button, TextView...)
Attribute	Define behavior and appearance
Namesapce	Identifiers Android attributes

XML Parser

- මේ XML Parser කියන කෙනාගෙන් කරන්නේ XML documents ටික read කරලා ඒක application එකට හේරුම් ගැනීමට සහ process කිරීමට හැකි format එකකට convert කරලා දෙන එක.
- Android වලදී ප්‍රධාන ආකාර 3කට මේ parser ගැන කතා කරන්න පුළුවන්



How does this work?



Android UI Configuration

R.java

R.java කියන file එක auto generate වෙන file එකක්. Project එකේ පාවිච්චි කරන සියලුම resources වලට reference එකක් හදාගන්නවා මේ class එක ඇතුළේදී. මේක generate කරලා දෙන්නේ Android Build system එකෙන්. මේ class එක XML resource එක සහ Java/Kotlin code එක අතර bridge එකක් විදිහට වැඩ කරනවා

- මේ R.java කියන class එක developersලා manually වෙනස් කරන්න අවශ්‍ය නෑ.

Purpose of R.java

- Provide unique integer IDs for all resources
- Allow developers to access XML resource programmatically
- Maintain strong connection between UI components and application logic

Resources included in R.java

- Layout File (R.layout)
- View IDs (R.id)
- String resources (R.string)
- Drawable resources (R.drawable)
- Color resources (R.color)
- Style resources (R.style)



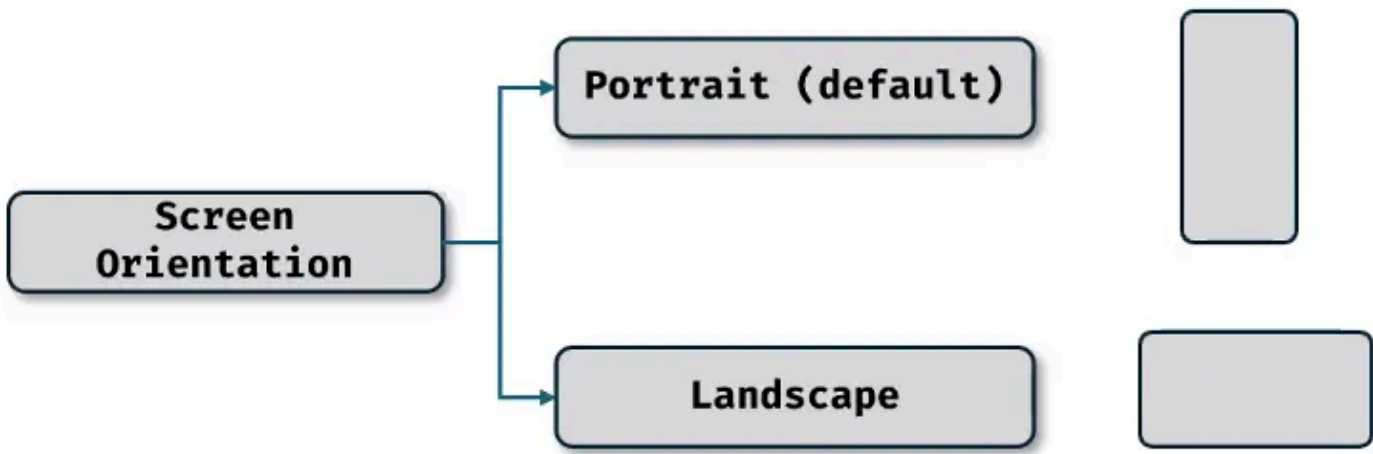
<Button
 android:id="@+id/submitBtn"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:text="@string/submit_btn_text"
 .../>

setContentView(R.layout.activity_main);
Button submitBtn = findViewById(R.id.submitBtn);

Android UI Configuration

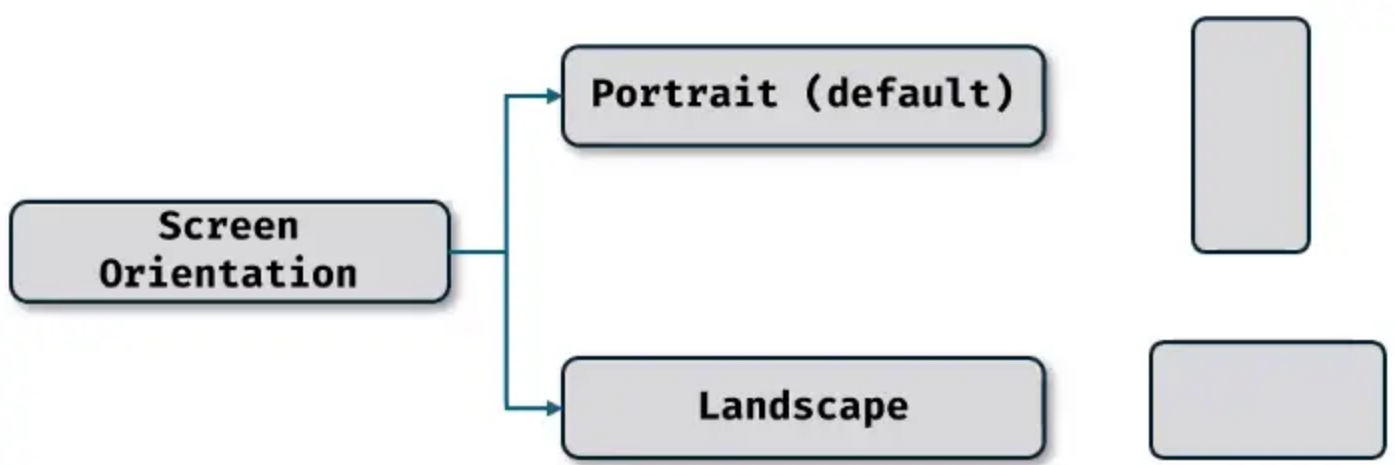
Screen Orientation

- The direction in which an Android Device screen is displayed



Screen Orientation

- මෙතැනදී අපි කතා කරන්නේ screen එක මොන දිශාවටද display කරන්න ඕනේ.



Default behavior

- When the device rotated, the activity is created
- UI components are reloaded
- Configuration changes occur automatically

Using AndroidManifest.xml



AndroidManifest.xml

```
<activity android:name=".activity.MainActivity"
  android:screenOrientation="landscape"/>
```

Using Java File



MainActivity.java

```
setRequestedOrientation(
  ActivityInfo.SCREEN_ORIENTATION_LANDSCAPE);
```

Screen Orientation Values

Value	Description
portrait	Locks screen in portrait mode
landscape	Locks screen in landscape mode
sensor	Follows device rotation
unspecified	Use system default

Hiding the Title Bar

Hiding the Title Bar using Theme/Style



style.xml

```
<style name="App_Style" parent="Theme.AppCompat.NoActionBar">
    ...
</style>
```



AndroidManifest.xml

```
<application
    ...
    android:theme="@style/App_Style">
```

Hiding the Title Bar programmatically



MainActivity.java

```
requestWindowFeature(Window.FEATURE_NO_TITLE);
setContentView(R.layout.activity_main);
Objects.requireNonNull(getSupportActionBar()).hide();
```

Full Screen Mode

Full Screen Mode



MainActivity.java

```
getWindow().setFlags(
    WindowManager.LayoutParams.FLAG_FULLSCREEN,
    WindowManager.LayoutParams.FLAG_FULLSCREEN
);
```

Application Running Lifecycle

- This explains how an Android application is created, executed, managed and terminated by the Android system.
- අපි සාමාන්‍ය දකින desktop application එකක් වගේ නෙවෙයි Android application එකක් වැඩ කරන්නේ. ඒ කියන්නේ desktop application එකක වගේ single entry point එකක් සහ single exit point එකක් නෑ.
- Android system එක විසින් application එකක් run කරන්නේ system resources, user interactions සහ component states මත පදනම් වෙලා.

Application Process States

- Android විසින් අපේ running වෙන applications වැදගත්කම අනුව states කිහිපයකට categories කරනවා.
 - a. Foreground Process (Currently interacting with the user) - user මේ වෙලාවේ interact වෙලා ඉන්න app එක. මෙයාට priority එක වැඩියි
 - b. Visible Process - app එක visible නමුත් focus වෙලා නෑ
 - c. Service Process (Running background services)
 - d. Background Process (App is not visible)
 - e. Empty process (No active components; kept for caching)
- Android may terminate lower-priority processes to free memory.

JVM vs Dalvik

Android විසින් ආවේනික JVM පාවිච්චි කරන්නේ නැහැ Application එකක් run කරන්න. ඒකට Dalvik virtual machine එක තමයි පාවිච්චි කරන්නේ. මේක හදල තියෙන විශේෂයෙන් mobile phone වලට

JVM (Java Virtual Machine)	Dalvik Virtual Machine (DVM)
Executes Java bytecode and is commonly used in desktop applications and server environment	This VM is designed specifically for Android to run application efficiently on devices with limited memory and processing power
Stack-based architecture	Register-based architecture
Executes <code>.class</code> files	Executes <code>.dex</code> (Dalvik Executable) files
Requires more memory	Optimized for low memory usage
Uses Java bytecode	Each Android app runs in its own Dalvik instance
Optimized for desktop / server systems	Faster execution for mobile devices

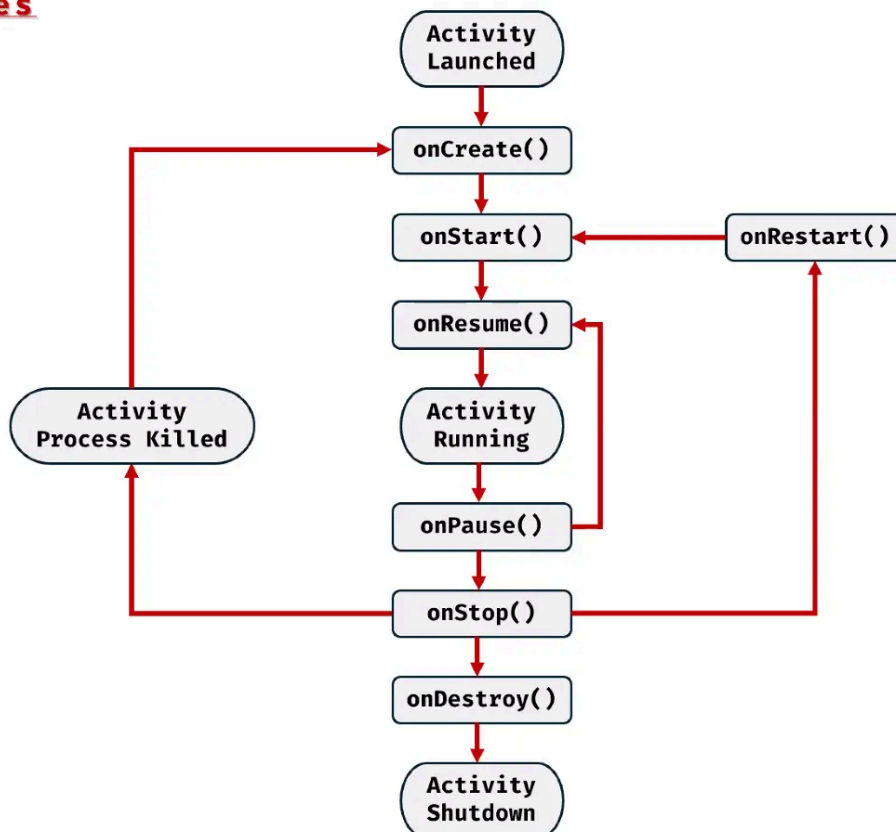
- Modern Android versions use ART (Android Runtime) instead of Dalvik

Activities

- user කෙනෙක් interact කරගන්න entry point එකක් විදිහට වැඩ කරන්නේ මෙන්න මේ activity එකක්
- හැම activity එකකටම Android system එක විසින් manage කරන lifecycle එකක් තියෙනවා
- සියලුම activity ගැන Androidmanifest.xml එකේ සඳහන් කරන්න ඕනේ
- අපි හදන application එකක multiple activities තියෙන්න පුළුවන් උනාට foreground එකේ එක මොහොතක active අවස්ථාවේ තියෙන්න පුළුවන් එක activity එකක් පමණයි

```
<activity android:name=".activity.HomeActivity" android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category android:name="android.intent.category.LAUNCHER"/>
    </intent-filter>
</activity>
```

Activities



- මේ lifecycle එකේ states වලට Android වලින් callback methods සපයල දීල තියෙනවා. අපිට පුළුවන් මේ methods override කරලා development වලට පාවිච්චි කරන්න පුළුවන්.

Lifecycle state / callback method	Description
-----------------------------------	-------------

onCreate()	• Called when the Activity is first created • Used to initialize UI elements • setContentView() is usually called here
onStart()	• Called when the Activity becomes visible to user • Activity is not yet ready for user interaction
onResume()	• Called when the Activity come to the foreground • User can now interact with Activity • Ideal place to start animation, sensors or listener
onPause()	• Called when another Activity partially covers the current one • Used save data, pause animations or unnecessary processes
onStop()	• Called when the Activity is no longer visible • Release heavy processes such as database connections
onRestart()	• Called when the Activity has been stopped and is about to start again
onDestroy()	• Called before the Activity destroyed • Used for final cleanup of resources

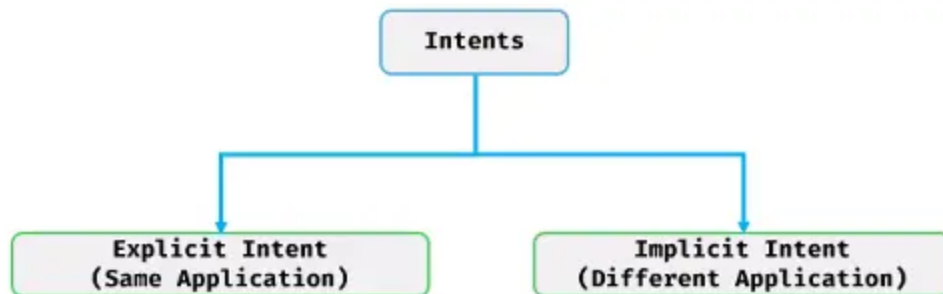
Intents

- An **intent** is a fundamental messaging object in Android
- Intent enable communication between different components of the same application or between different applications
- In simple terms, an Intent tells the Android System: "I want to do something"

Uses of Intents

Intents are commonly used to:

- Start an **Activity**
- Start a **Service**
- Deliver a **Broadcast**
- Pass data between components



Explicit Intent

- මේ explicit intent පාවිච්චි කරන්නේ target component (Activity or Service) පැහැදිලිව දන්නවා නම් පමණි
- පටන් ගැන්ම සඳහා විශේෂයෙන් exact class name එකක් පාවිච්චි කරයි
- same application එකේ නමයි බොහෝ දුරට පාවිච්චි කරන්නේ
- More secure and predictable
- Mostly used for internal navigation

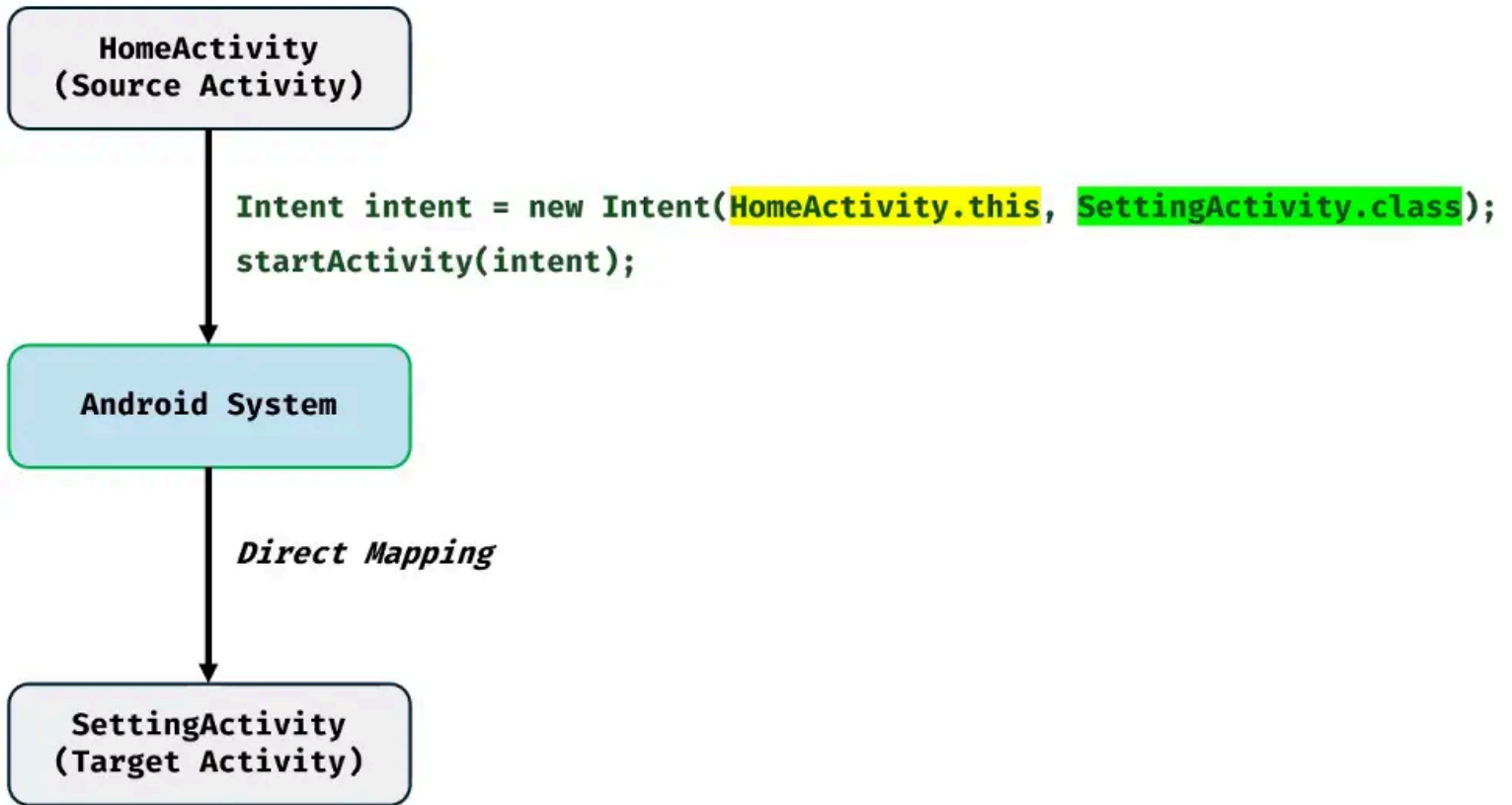
Explicit Intent Calling

```
Intent intent = new Intent(HomeActivity.this, SettingActivity.class);
startActivity(intent);
```

Explicit Intent Calling With Data

```
Intent intent = new Intent(HomeActivity.this, SettingActivity.class);
intent.putExtra("username", "Sahan");
startActivity(intent);
-----
String username = getIntent().getStringExtra("username");
```

Explicit Intent Working Flow



Implicit Intent

- මේ implicit intent කියලා කියන්නේ target component එක විශේෂයෙන් දන්නේ නැති අවස්ථාවකදී.
- Intent එකේ තියෙන action එක, data හා category මත Android system එක විසින් target component එක තෝරාගනු ලබයි.
- Target component එකේ නම විශේෂයෙන් සඳහන් නොකරයි
- මෙතැනදී intent filters පාවිච්චි කරයි
- වෙනත් application එකක component එකක් start එකක් විදිහට ගන්න පුලුවන්
- multiple apps වලින් එකක් තෝරාගැනීමට userට සිදුවිය හැක

Opening a Web Page

```
Intent intent = new Intent(Intent.ACTION_VIEW);
intent.setData(Uri.parse("http://www.google.com")); // Uri ==> android.net.Uri
startActivity(intent);
```

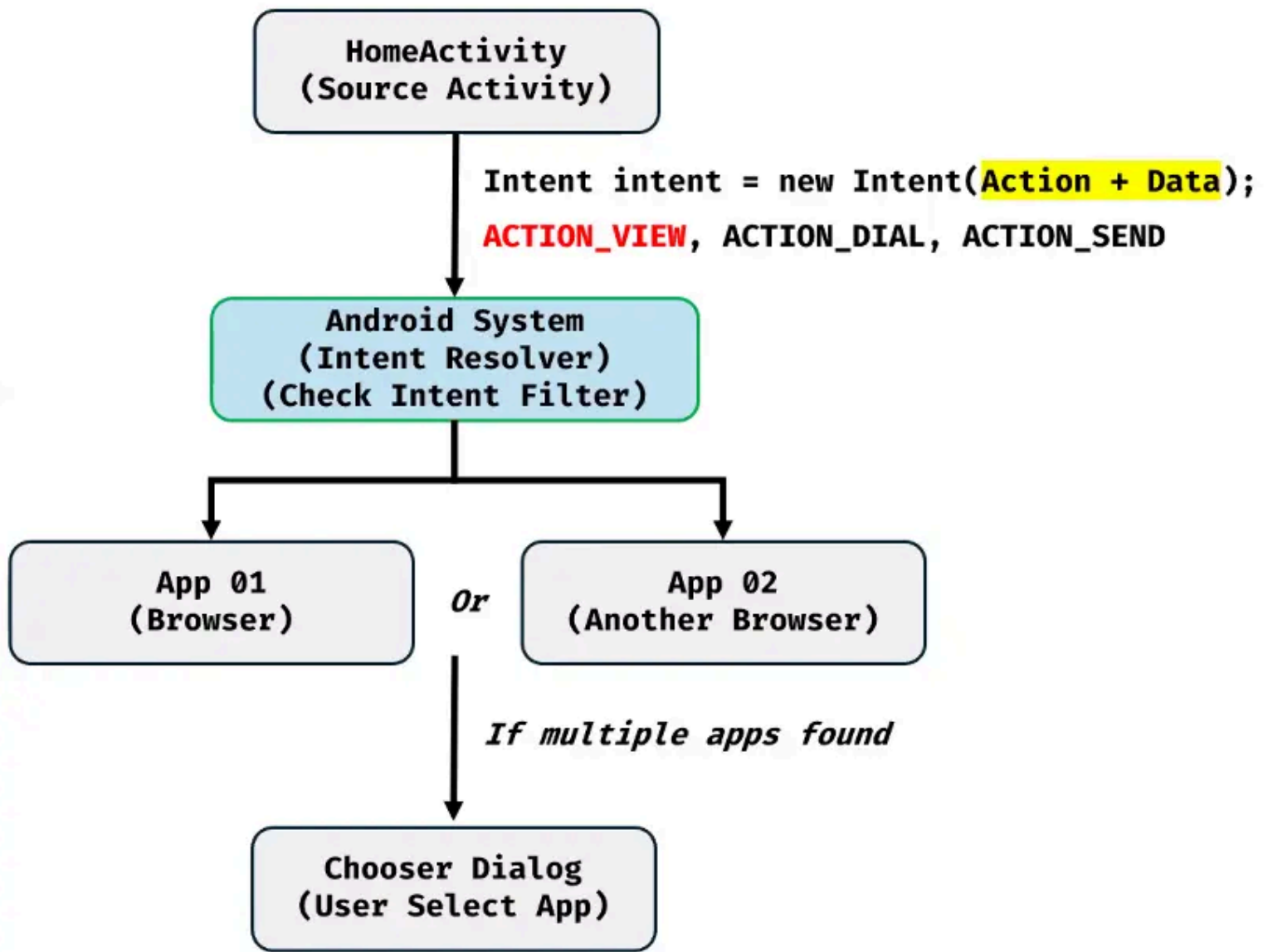
Dialing a Phone Number

```
Intent intent = new Intent(Intent.ACTION_DIAL);  
intent.setData(Uri.parse("tel:0771234567"));  
startActivity(intent);
```

Sharing Text

```
Intent intent = new Intent(Intent.ACTION_SEND);  
intent.setType("text/plain");  
intent.putExtra(Intent.EXTRA_TEXT, "Hello Ramin");  
startActivity(Intent.createChooser(intent, "Share via"));
```

Implicit Intent Working Flow



Prevent duplicate instance when calling Intent

අපි intent එකක් හරියට manage කරලා තිබ්බේ නැත්නම් එකම activity එකෙන් multiple instances හැදෙන්න පුළුවන්. ඒ වගේ අවස්ථාවකදී,

- memory නාස්තිය වැඩියි.
- Confusing navigation behavior
- Unexpected back stack issues
- Poor user experience

Why duplicate Activity instances occur?

Android වලදී default පාවිච්චි කරන්නේ standard launch mode එක.

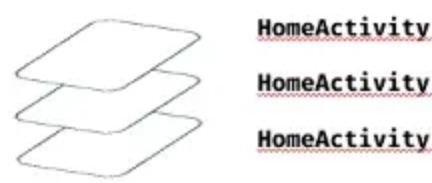
Default Behavior:

- startActivity() is called every time
- Android creates a new instance of the target activity even if that Activity is already running

Android Task and Back Stack Overview

- **Task** → A collection of Activities arranged in a stack
- **Back Stack** → Last Activity is at the top
- Pressing back button pops the top activity

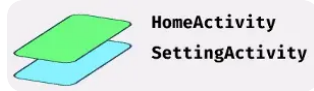
If duplicate instance exist:



Pressing back feels "Stuck" in the same activity

Methods to Prevent Duplicate Activity Instances

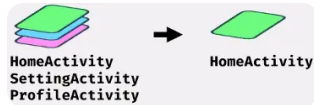
Method	Type	Description	Example code
Using launch mode in Android manifest. xml	standard (default)	• Always create a new instance • Duplicate instance allowed	<pre><activity android:name=".activity.Setting Activity"/></pre>
	singleTop	• If Activity is already at the top, no new instance is created • onNewIntent() is called	<pre><activity android:name=".activity.Setting Activity" android:launchMode="singleTop" / ></pre>



HomeActivity
(top) → reused

singleTask

- Only one instance exists in the task
- If exists, it is brought to front
- All activities above it are destroyed



```
<activity
    android:name=".activity.Setting
    Activity"
    android:launchMode="singleTask"
/>
```

singleInstance

- Activity runs in its own task
- No other Activities allowed in that task

```
<activity
    android:name=".activity.Setting
    Activity"
    android:launchMode="singleInsta
    nce"/>
```

Using Intent
Flags
(Runtime
Control)

FLAG_ACTIV
ITY_CLEAR_
TOP

- If Activity exists → removes all Activities above it
- Calls onNewIntent()

```
Intent intent = new
Intent(this,MainActivity.class)
;
intent.addFlags(Intent.FLAG_ACT
IVITY_CLEAR_TOP);
startActivity(intent);
```

FLAG_ACTIV
ITY_SINGLE
_TOP

- Works like singleTop
- Only prevents duplicates if Activity is already on top

```
intent.addFlags(Intent.FLAG_ACT
IVITY_SINGLE_TOP)
```

	Combining Flags	• Most commonly used combination • Safe and flexible	<pre>intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP Intent.FLAG_ACTIVITY_SINGLE_TOP)</pre>
Handling onNewIntent()	onNewIntent()	• When an existing Activity instance is reused, data comes through onNewIntent()	<pre>@Override protected void onNewIntent(Intent intent){ super.onNewIntent(intent); setIntent(intent); // Update Intent }</pre>
Finishing Current Activity	finish()	• Prevents returning to previous instance	<pre>startActivity(); finish();</pre>

Intent Filters

- මේ intent filters අපි add කරන්නේ Androidmanifest.xml file එකේ
- මේ intent filters වලින් කරන්නේ Android system එකට කියනවා මොන වගේ intent එකකටද පුළුවන් අපි ළඟ තියෙන activity, service හෝ BroadcastReceiver වලටද ප්‍රතිචාර දක්වන්න පුළුවන්
- සරලව කිව්වොත් intent එකක් කියලා කියන්න request එකක් නම් intent filter එකක් කියලා කියන්නේ ඒ request එක handle කරන්න පුළුවන් කාටද, එයට rules ටිකක් පනවන එක
 - Intent → Request
 - Intent-filter → rule that decides who can handle that request

Why Intent Filters are needed

- implicit intents වලට අවසර ලබා දීම
- අපේ app එකත් එක්ක වෙනත් app වලට සම්බන්ධ වීමට අවස්ථාවක් ලබා දීම

- To define launcher activity
- To handle system actions (dial, share, view URL, open Files....)

```

<activity
    ...>
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>

```

They can be used with

- <activity>
- <service>
- <receiver>

Main components of Intent Filters

Component	Example	Description
<action>	<action android:name="android.intent.a ction.MAIN" />	Defines what action the intent wants to perform Common actions: • ACTION_VIEW • ACTION_SEND • ACTION_DIAL • ACTION_MAIN
<category>	<category android:name="android.intent.c ategory.DEFAULT" />	Defines who can handle the intent Important categories: • DEFAULT → Requires for implicit intents

		• LAUNCHER → App launcher entry • BROWSABLE → For web links
<code><data></code>	<code><data android:scheme="https"/></code>	Defines data type or URI the intent works with Data attributes • scheme → http, https, tel, geo • host → domain name • mimeType → text/plain, image/* • Path, pathPrefix

Intent වැඩ කරද්දී Android වලින් check කරන කරුණු තුනක් තියෙනවා

- action එක match වෙනවද කියන එක
- category එක match වෙනවද කියන එක
- data එක match වෙනවද කියන එක

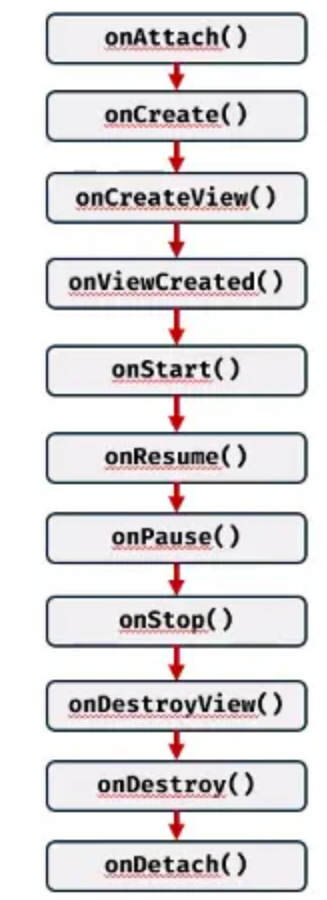
මේ එකක් හරි fail උනොත් intent එක reject කරනවා

Fragments

Key Characteristics

- Fragment එකක් හැම වෙලාවේම වැඩ කරන්නේ Activity එකක් මත
- Single Activity එකකට multiple fragments තියෙන්න පුළුවන්
- Fragments have their own lifecycle, which is closely related to the activity lifecycle
- Fragments improve **code reusability** and **UI flexibility**

Fragment Lifecycle



Callback method	Description
onAttach()	Fragment attach to the activity
onCreate()	Fragment created
onCreateView()	Layout inflated (UI created)
onViewCreated()	View ready
onStart()	Fragment visible
onResume()	Fragment active
onPause()	Fragment partially visible
onStop()	Fragment hidden
onDestroyView()	View destroyed
onDestroy()	Fragment destroyed

onDetach()

Fragment detach from activity

Fragment Implementation - XML Mapping



fragment_home.xml

```
<androidx.constraintlayout.widget.ConstraintLayout ...>

    <TextView
        ...
        android:text="@string/home_fragment_txt"
        .../>
</androidx.constraintlayout.widget.ConstraintLayout>
```



HomeFragment.java

```
public class HomeFragment extends Fragment {
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
        @Nullable ViewGroup container,
        @Nullable Bundle savedInstanceState) {
        return inflater.inflate(R.layout.fragment_home,
            container,
            false);
    }
}
```



activity_home.xml

```
<androidx.constraintlayout.widget.ConstraintLayout ...>

    <androidx.fragment.app.FragmentContainerView
        ...
        android:name="lk.raminsenanayake.fragments.fragment.HomeFragment"
        ...
        tools:layout="@layout/fragment_home" />

    <Button
        .../>
</androidx.constraintlayout.widget.ConstraintLayout>
```



HomeActivity.java

```
public class HomeActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_home);
    }
}
```

```
public View onCreateView(@NonNull LayoutInflater inflater,
    @Nullable ViewGroup container,
    @Nullable Bundle savedInstanceState) {
    return inflater.inflate(R.layout.fragment_home, container, false);
}
```

Parameter

Description

LayoutInflater inflater

- Used to convert XML layout files into view object

<code>ViewGroup container</code>	• This is the parent view where the fragment UI will be attached • Usually, this is the <code>FragmentContainerView</code> in the Activity layout
<code>Bundle savedInstanceState</code>	• Contains previously used fragment state • Used when: → Screen rotation happens → Fragment is recreated by the system

Parameter	Description
<code>R.layout.fragment_home</code>	XML layout file for the Fragment
<code>container</code>	Parent view (<code>FragmentContainerView</code>)
<code>false (attachToRoot)</code>	Do NOT attach the view immediately

attachedToRoot = false



attachedToRoot = true



- Fragment `onCreateView()` → always **false**
- Activity / manual UI addition → true for immediate attach)

Fragment Implementation - Java Mapping



fragment_setting.xml

```
<FrameLayout ...
    tools:context=".fragment.SettingFragment">

    <androidx.constraintlayout.widget.ConstraintLayout
        ...>
        ...
    </androidx.constraintlayout.widget.ConstraintLayout>

</FrameLayout>
```



SettingFragment.java

```
public class SettingFragment extends Fragment {
    @Nullable
    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
        Bundle savedInstanceState) {
        return inflater.inflate(R.layout.fragment_setting, container, false);
    }
}
```



activity_home.xml

```
<androidx.constraintlayout.widget.ConstraintLayout ...>

    <androidx.fragment.app.FragmentContainerView
        ...
        android:id="@+id/fragment_container_view" />

    <Button
        .../>

</androidx.constraintlayout.widget.ConstraintLayout>
```



HomeActivity.java

```
public class HomeActivity extends AppCompatActivity {  
    private Button settingBtn;  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        ...  
        Button settingBtn = findViewById(R.id.settingBtn);  
        settingBtn.setOnClickListener(new View.OnClickListener() {  
            @Override  
            public void onClick(View view) {  
                getSupportFragmentManager().beginTransaction()  
                    .setReorderingAllowed(true)  
                    .replace(R.id.fragment_container_view,  
SettingFragment.class,null)  
                    .commit();  
            }  
        });  
    }  
}
```

Layouts

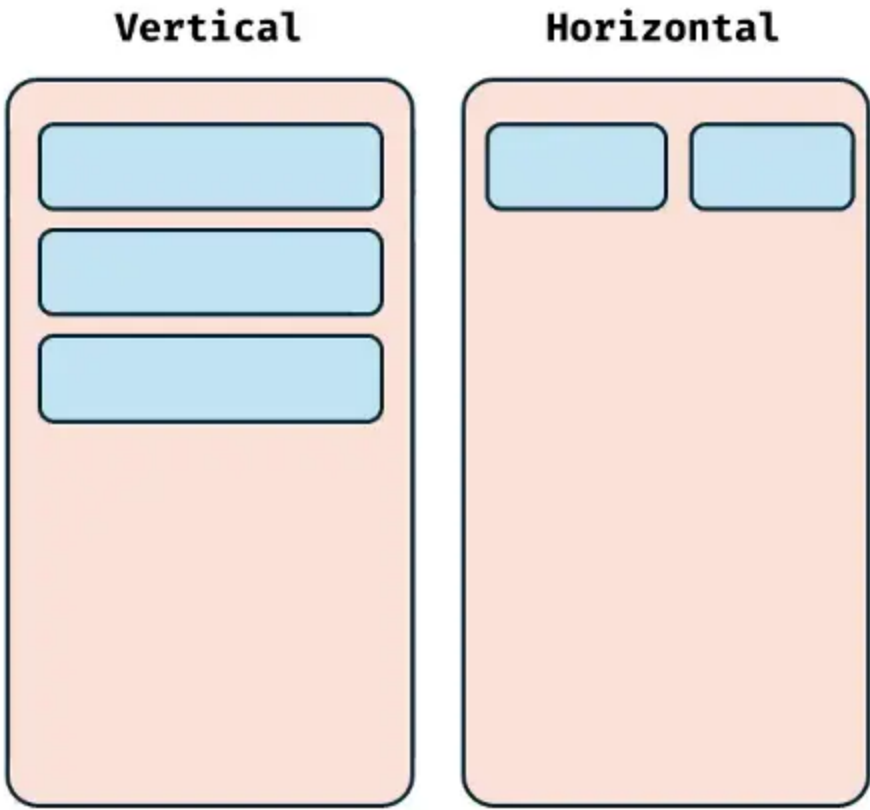
- A layout is a ViewGroup that defines how UI components (Views) are arranged on the screen

LinearLayout

- එක් දිශාවකට පමණක් child views arrange කරනු ලබයි
 - Vertical
 - Horizontal

Attribute	Description
android:orientation	vertical / horizontal

<code>android:gravity</code>	Aligns child views
<code>android:layout_gravity</code>	aligns the view inside parent
<code>android:layout_weight</code>	Distributes extra space



RelativeLayout

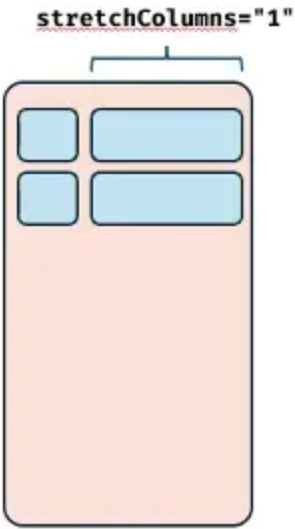
- parentට හෝ අනෙක් views වලට සාපේක්ෂව child views position කරන්න මේ layout එක පාවිච්චි කරන්න පුළුවන්

Attribute	Description
<code>android:layout_below</code>	Below another view
<code>android:layout_above</code>	Above another view
<code>android:layout_toRightOf</code>	Right of another view
<code>android:layout_alignParentTop</code>	Align to parent

<code>android:layout_centerInParent</code>	Center in parent
--	------------------

TableLayout

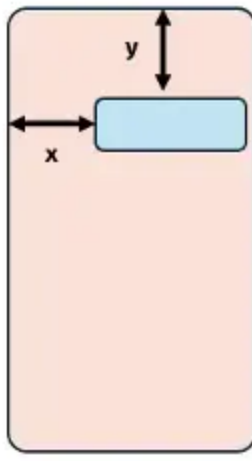
- අපි පාවිච්චි කරන views ටික rows හා columns විදිහට manage කරනවා HTML table එක වගේ
 - Each row is represented by `TableRow`
 - Columns adjust automatically



Attribute	Description
<code>android:stretchColumns</code>	Stretch column with (Avoid extra spaces, start with 0 - column number)
<code>android:shrinkColumns</code>	Shrink column width (Avoid overflow)

AbsoluteLayout

- x හා y coordinates වලින් views වල position හඳුනා ලබයි
- මේ layout එක deprecate වෙලා තියෙන්නේ
- Attributes:
 - `android:layout_y` → X coordinate
 - `android:layout_x` → Y coordinate



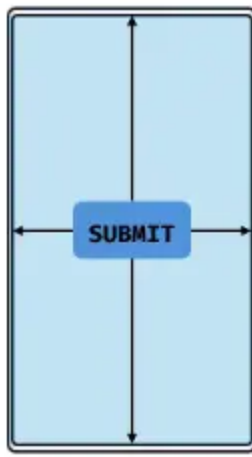
FrameLayout

- Single view එකක් display කරන්න හෝ එකක් මත එකක් views තබා ගැනීමට මේ layout එක use කරන්න පුළුවන්



ConstraintLayout

- Modern Android UI development වලදී ConstraintLayout එක පාවිච්චි කරනු ලබයි
- Parentට හෝ අනිත් components වලට සාපේක්ෂව UI components position කරගන්න constraints පාවිච්චි කරන්න පුළුවන් (nested multiple layouts පාවිච්චි කරන්න ඕනේ නෑ)



Attribute	Description
<code>app:layout_constraintStart_toStartOf</code>	Align start
<code>app:layout_constraintEnd_toEndOf</code>	Align end
<code>app:layout_constraintTop_toTopOf</code>	Align top
<code>app:layout_constraintBottom_toBottomOf</code>	Align bottom

Event Handling

- UI component එක්ක user කරන interaction detect කරගෙන ඒවාට respond එකක් දීමට පාවිච්චි කරන mechanism එක නමයි මේ event handling වල අපි කතා කරන්නේ
- User interactions (Event) include:
 - Button clicks
 - Long press
 - Touch gesture
 - Key press
 - Focus changes
 - Menu selections
 - Drag and Drop

Android uses **Listener Interfaces** to handle these events

Event Handling Structure

```
view.setOnEventListener(  
    new View.OnEventListener() {  
        @Override  
        public void onEvent(View view) {  
            // event code - do something  
        }  
    }  
);
```

```
view.setOnEventListener(v -> {  
    // event code - do something  
});
```

Lamda expression

Method	Definition	Listener and method	Example
<code>onClick()</code>	Triggered when the user taps (Click) a view	Interface → <code>OnClickListener</code> Method → <code>void onClick(View v){}</code>	<pre>Button onClickBtn = findViewById(R.id.onClickBtn); onClickBtn.setOnClickListener(new View.OnClickListener() { @Override public void onClick(View view) { ... } });</pre>
<code>onLongClick()</code>	Triggered when the user presses and holds a view	Interface → <code>View.OnLongClickListener</code> Method → <code>boolean onLongClick(View v){}</code>	<pre>longPressBtn.setOnClickListener(new View.OnClickListener() { @Override public void onClick(View view) { ... return true; } });</pre> True → Trigger only long click False → Trigger long click + click

`onFocusChange()`

Triggered when a view gains or loses focus

Interface → `View.OnFocusChangeListener`

Method → `void onFocusChange(View v, boolean hasFocus){}`

```
EditText editText =
    findViewById(R.id.editText);

editText.setOnFocusChangeListener(new
    View.OnFocusChangeListener() {
        @Override
        public void onFocusChange(View
            view, boolean b) {
            if (b){
                view.setBackgroundColor(Color.LTGRAY); // Color ==>
                android.graphics
            }
        }
    });
```

`onKey()`

Triggered when a hardware or software key pressed

Interface → `View.OnKeyListener`

Method → `boolean onKey(View v, int keycode, KeyEvent event){}`

```
editText.setOnKeyListener(new
    View.OnKeyListener() {
        @Override
        public boolean onKey(View
            view, int i, KeyEvent
            keyEvent) {
            if (keyEvent.getAction() ==
                KeyEvent.ACTION_DOWN && i ==
                KeyEvent.KEYCODE_ENTER){
                ...
            }
            return false;
        }
    });
```

onTouch()	Handles touch events like press, move and release	Interface → View.OnTouchListener ner Method → boolean onTouch(View v, MotionEvent me){} Touch Activities: ACTION_DOWN , ACTION_UP , ACTION_MOVE	<pre>View view = findViewById(R.id.view); view.setOnTouchListener(new View.OnTouchListener() { @Override public boolean onTouch(View view, MotionEvent motionEvent) { if (motionEvent.getAction() == MotionEvent.ACTION_DOWN) { ... } return false; } });</pre>
onCreateContextMenu()	Used to create context menu when a view is long-pressed	Method → void onCreateContextMenu(ContextMenu menu, View v, ContextMenu.ContextMenuInfo menuInfo){}	<pre>registerForContextMenu(view); @Override public void onCreateContextMenu(ContextMenu menu, View v, ContextMenu.ContextMenuInfo menuInfo) { super.onCreateContextMenu(menu, v, menuInfo); menu.setHeaderTitle("Options"); menu.add(0,v.getId(),0,"Edit"); menu.add(0,v.getId(),1,"Edit"); } menu.add(int groupId, int itemId, int orderId, Charsequence title) </pre>

onDrag()	Triggered when a view is dragged	Interface → View.OnDragListener Method → boolean onDrag(View v, DragEvent event) {} Drag Actions: ACTION_DROP , ACTION_DRAG_START ED , ACTION_DRAG_END	<pre>view.setOnDragListener(new View.OnDragListener() { @Override public boolean onDrag(View view, DragEvent dragEvent) { if (dragEvent.getAction() == DragEvent.ACTION_DROP){ ... } return true; } });</pre>
----------	----------------------------------	--	---

Android UI Components

AutoCompleteTextView

- AutoCompleteTextView එක කියන්නේ editable text field එකක්. මේකේ විශේෂත්වය නමයි user විසින් type කරනකොට automatically suggestions කරගන්න හදාගන්න පුළුවන්
- මේ වගේ component එකක් search box එකක් විදිහට හෝ city / country selection එකක් විදිහට use කරන්න පුළුවන්



Adapter

- අපි ලබාදෙන array එකක් හෝ list එකක් Android UI component වලට direct read කරන්න බෑ. ඒ නිසා adapter එකක් පාවිච්චි කරනවා. ඒකෙන් කරන්නේ data, views විදිහට convert කරන එක

Adapter	Usage
ArrayAdapter	Simple List
BaseAdapter	Custom layout
CursorAdapter	Database data

{ "Ampara", "Colombo", "Gampaha" }

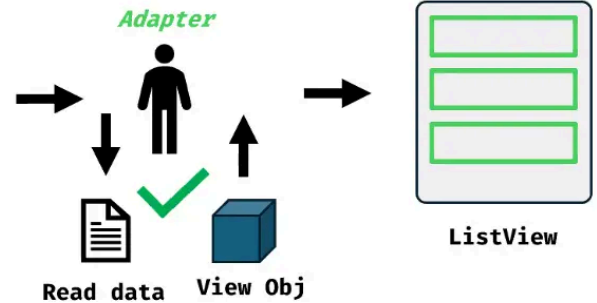
- Ampara
- Colombo
- Gampaha



ListView

{ "Ampara", "Colombo", "Gampaha" }

- Ampara
- Colombo
- Gampaha

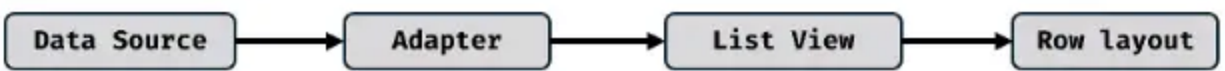


XML Layout	Java
<pre> <AutoCompleteTextView android:id="@+id/autoComple eTextView" ... android:completionThreshold= "1" android:hint="Type Here" .../> </pre>	<pre> AutoCompleteTextView autoCompleteTextView = findViewById(R.id.autoCompleteTextView); String[] cities = {"Ampara", "Colombo", "Anuradhapura", "Gampaha", "Galle", "Kurunegala", "Kegalle", "Bandarawe la", "Bandaragama", "Badulla", "Mathale", "Mata ra", "Hangolla", "Hanpitiya"}; ArrayAdapter<String> arrayAdapter = new ArrayAdapter<>(this, android.R.layout.simple_dropdown_item_1line , cities); autoCompleteTextView.setAdapter(arrayAdapte r); </pre>

- `completionThreshold` → number of characters before suggestions appear
- `android.R.layout.simple_dropdown_item_1line` → Pre-defined row layout file

ListView

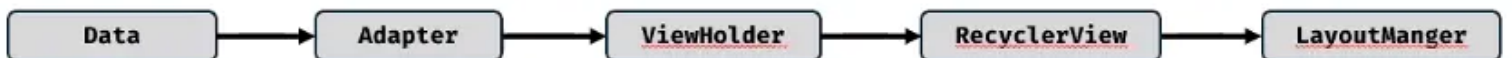
- Vertically scroll කරන්න පුළුවන් items නියෝන component එකක් නමයි `ListView` කියන්නේ



XML Layout	Java
<pre> <ListView android:id="@+i d/listView" .../> </pre>	<pre> ListView listView = findViewById(R.id.listView); String[] languages = {"Java", "Kotlin", "SQL", "PHP", "Python", "JavaScript"}; ArrayAdapter<String> arrayAdapter = new ArrayAdapter<> (ListViewActivity.this, android.R.layout.simple_list_item_1, languages); listView.setAdapter(arrayAdapter); listView.setOnItemClickListener(new AdapterView.OnItemClickListener() { @Override public void onItemClick(AdapterView<?> adapterView, View view, int position, long id) { String lang = languages[position]; Toast.makeText(ListViewActivity.this, lang, Toast.LENGTH_S HORT).show(); } }); </pre>

RecyclerView

- **RecyclerView** කියන්නේ list view එක වෙනුවට advanced හා optimize කරපු component එකක්
- Used for:
 - Large data set
 - Complex layouts
 - High-performance scrolling



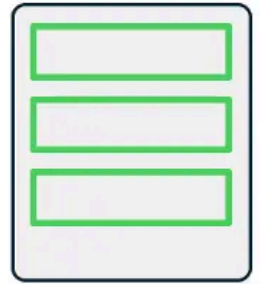
RecyclerView.Adapter

- Connect data to UI



ViewHolder

- Holds references item views
- Improve performance



RecyclerView

- Control item layout
- Linear/ Grid/ StaggeredGridLayout

LayoutManager



Dependency

```
implementation 'androidx.recyclerview:recyclerview:1.3.2'
```

XML Layout - activity_main.xml

```
1 <androidx.recyclerview.widget.RecyclerView
2     android:id="@+id/recyclerView"
3     .../>
```

XML Layout - item_row.xml

```
1 <androidx.constraintlayout.widget.ConstraintLayout ...>
2     <TextView
3         android:id="@+id/textView"
4         ... />
5 <androidx.constraintlayout.widget.ConstraintLayout .../>
```

```
1 public class LangAdapter extends RecyclerView.Adapter<LangAdapter.ViewHolder> {
2     String[] data;
3
4     public LangAdapter(String[] data) {
5         this.data = data;
6     }
7
8     @NonNull
9     @Override
10    public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
11        View view = LayoutInflater.from(parent.getContext())
12            .inflate(R.layout.item_row, parent, false);
13        return new ViewHolder(view);
14    }
15
16    @Override
17    public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
18        holder.textView.setText(data[position]);
19    }
20
21    @Override
22    public int getItemCount() {
23        return data.length;
24    }
25
26    public static class ViewHolder extends RecyclerView.ViewHolder {
27        TextView textView;
28
29        public ViewHolder(@NonNull View itemView) {
30            super(itemView);
31            textView = itemView.findViewById(R.id.textView);
32        }
33    }
34 }
```

MainActivity.java

```

1 RecyclerView recyclerView = findViewById(R.id.recyclerView);
2 recyclerView.setLayoutManager(new LinearLayoutManager(this));
3
4 String[] langData = {"Java", "Python", "Kotlin"};
5 LangAdapter langAdapter = new LangAdapter(langData);
6 recyclerView.setAdapter(langAdapter);

```

```

public interface OnCityItemClickListener { 1
    void onCityItemClick(String value);
}

```

```

public class CityAdapter extends RecyclerView.Adapter<CityAdapter.ViewHolder>
{
    ...
    4 private final OnCityItemClickListener onCityItemClickListener;
    5 public CityAdapter(String[] values, OnCityItemClickListener itemClick) {
        ...
        6 this.onCityItemClickListener = itemClick;
    }
    ...
    @Override
    public void onBindViewHolder(@NonNull ViewHolder holder, int position) {
        String value = data[position];
        ...
        holder.button.setOnClickListener(new View.OnClickListener() {
            ...
            7 onCityItemClickListener.onCityItemClick(value);
            ...
        });
    }
    ...
}

```

```

public class AdRecyclerViewActivity extends AppCompatActivity
    implements OnCityItemClickListener { 2

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        ...
        8 CityAdapter cityAdapter = new CityAdapter(cities, this);
        ...
    }

    3 @Override
    public void onCityItemClick(String value) {
        Toast.makeText(this, value, Toast.LENGTH_SHORT).show();
    }
}

```

WebView

XML Layout

```

<WebView
    android:id="@+id/webView"
    ... />

```



AndroidManifest.xml

```
<manifest>
    <uses-permission android:name="android.permission.INTERNET" />
    <application>
        ...
    </application>
</manifest>
```



Java Source

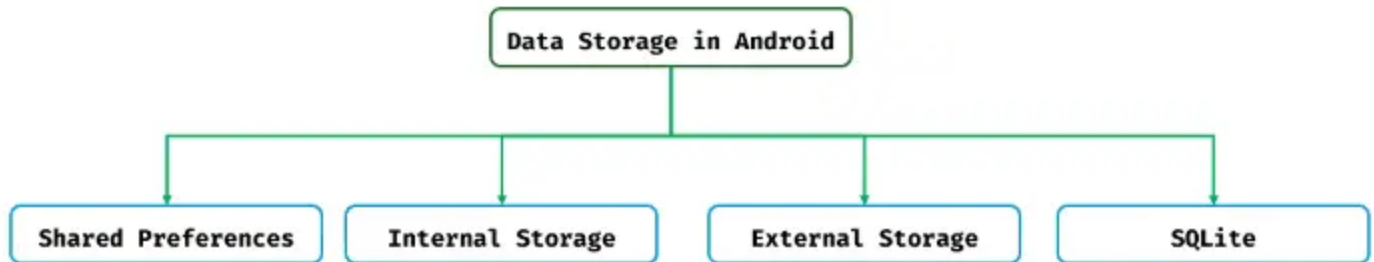
```
public class WebViewActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        ...
        WebView webView = findViewById(R.id.webView);
        webView.getSettings().setJavaScriptEnabled(true); // Enable JS
        webView.setWebViewClient(new WebViewClient()); // Prevent open in
default browser
        webView.loadUrl("https://www.google.com"); //url

        getOnBackPressedDispatcher().addCallback(this, new
OnBackPressedCallback(true) {
            @Override
            public void handleOnBackPressed() {
                if (webView.canGoBack()) {
                    webView.goBack();
                } else {
                    finish();
                }
            }
        });
    }
}
```

Data Storage

- Data storage කියලා මෙනෙදි සඳහන් කරන්නේ application එකක data; store කරන්න, retrieve කරන්න හා manage කරන්න පාවිච්චි කරන ක්‍රමවේදයන් ගැන.
- Android වලදී data size, sensitivity සහ usage එක අනුව data store කරන්න ක්‍රම කීපයක් පාවිච්චි කරයි.



Shared Preferences

- Shared preferences පාවිච්චි simple data ප්‍රමාණයක් store කරගන්න
- Data store කරගන්නේ **key-value** pair එකක් විදිහට (Ex: `isLoggedIn = true`)
- `String` , `int` , `boolean` , `float` , `long` are supported
- Store in private directory

```
private static final String APP_DATA = "my_app_data";
...
SharedPreferences sharedPreferences = getSharedPreferences(APP_DATA, MODE_PRIVATE);
SharedPreferences.Editor editor = sharedPreferences.edit();
editor.putBoolean("isLoggedIn", true);
// editor.commit(); /// immediately
editor.apply(); /// asynchronously
```

```
<map>
  <boolean name="isLoggedIn" value="true" />
</map>
```

lk/jiat/data_storage/shared_prefs/my_app_data.xml

Store

lk/jiat/data_storage/shared_prefs/my_app_data.xml

<map>

<boolean name="isLoggedIn" value="true" />

</map>

retrieve

```
SharedPreferences sharedPreferences = getSharedPreferences(APP_DATA, MODE_PRIVATE);  
boolean isLoggedIn = sharedPreferences.getBoolean("isLoggedIn", false); /// key, default_value
```

Retrieve

Internal Storage

- Internal storage එකේදී data store වෙන්නේ app එකේ private memory එකේ.
- app එක uninstall කරද්දී අයිත් වෙලා යනවා.

```
private void storeData() {  
    // getFilesDir(); /// Files directory -> lk/jiat/data_storage/files  
    // getCacheDir(); /// cache directory -> lk/jiat/data_storage/cache  
    try {  
        // File file = new File(getFilesDir(), "AppData.txt"); /// ->  
        lk/jiat/data_storage/files/AppData.txt  
        FileOutputStream fileOutputStream = openFileOutput("AppData.txt",  
MODE_PRIVATE);  
        String text = "My name is Ramin";  
        fileOutputStream.write(text.getBytes());  
        fileOutputStream.flush();  
        fileOutputStream.close();  
    } catch (IOException e) {  
        throw new RuntimeException(e);  
    }  
}
```

```

private void readData() {
    try {
        FileInputStream fileInputStream = openFileInput("AppData.txt");
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(fileInputStream));
        StringBuilder sb = new StringBuilder();
        String line;
        while ((line = bufferedReader.readLine()) != null) {
            sb.append(line);
        }
        fileInputStream.close();
        bufferedReader.close();
        Toast.makeText(this, sb.toString(), Toast.LENGTH_SHORT).show();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

```

External Storage

- outside of the app's internal memory
- it can be visible to user (File Manager)
- External storage
 - app specific external storage(Android/data/...) → Removed when app uninstalled
 - public directory (Downloads, Documents,DCIM) → Files may remain after app uninstall

App specific external storage

```

private static final String FILE_NAME = "data.txt";

```

```

private void writeFile() {
    try {
        File dir = getExternalFilesDir(null);
        File file = new File(dir, FILE_NAME);
        FileOutputStream fileOutputStream = new FileOutputStream(file);
        String text = "My name is Ramin";
        fileOutputStream.write(text.getBytes());
        fileOutputStream.close();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

```

```

private void readFile() {
    try {
        File dir = getExternalFilesDir(null);
        File file = new File(dir, FILE_NAME);
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(new FileInputStream(file)));
        StringBuilder stringBuilder = new StringBuilder();
        String line;
        while ((line = bufferedReader.readLine()) != null) {
            stringBuilder.append(line);
        }
        bufferedReader.close();
        Toast.makeText(this, stringBuilder.toString(), Toast.LENGTH_SHORT).show();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

```

Public Directory

```

private static final String FILE_NAME = "data.txt";

```

```

private void writeFileInDownloads() {
    try {
        ContentValues values = new ContentValues();
        values.put(MediaStore.Downloads.DISPLAY_NAME, FILE_NAME);
        values.put(MediaStore.Downloads.MIME_TYPE, "text/plain");

        Uri uri =
getContentResolver().insert(MediaStore.Downloads.EXTERNAL_CONTENT_URI, values);
        OutputStream outputStream = getContentResolver().openOutputStream(uri);
        String text = "This is sample text";
        outputStream.write(text.getBytes());
        outputStream.flush();
        outputStream.close();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

```

Sometimes we need to manage permissions to run in old devices. (AndroidManifest and Runtime Permissions) when use older API levels.

Access file using File Picker and read data

```

/// open file picker
private void openFilePicker() {
    Intent intent = new Intent(Intent.ACTION_OPEN_DOCUMENT);
    intent.setType("*/*");
    intent.addCategory(Intent.CATEGORY_OPENABLE);
    startActivityForResult(intent, 101); // deprecated
}

```

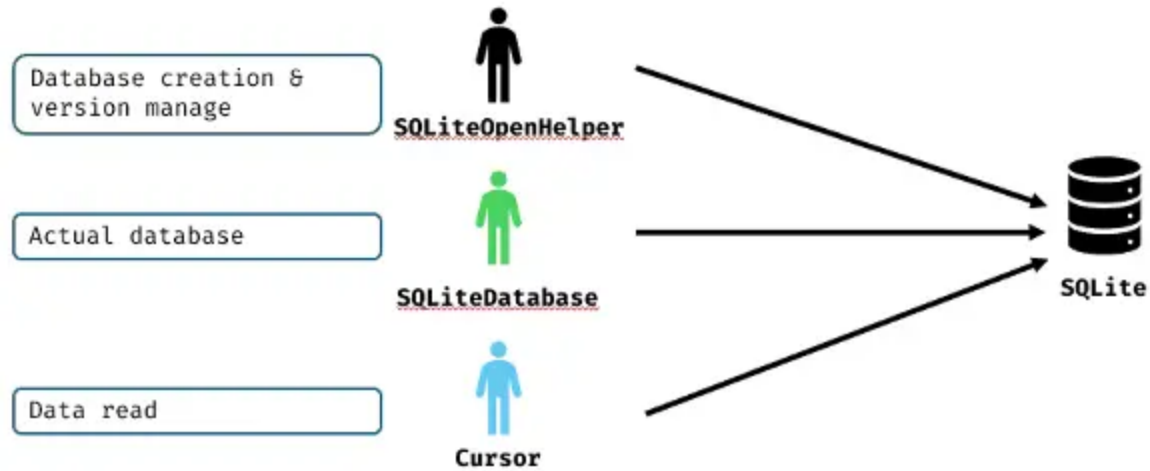
```

@Override
protected void onActivityResult(int requestCode, int resultCode, @Nullable Intent data)
{
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode == 101 && resultCode == RESULT_OK) {
        if (data != null) {
            Uri uri = data.getData();
            try {
                InputStream inputStream = getContentResolver().openInputStream(uri);
                BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(inputStream));
                String line;
                StringBuilder sb = new StringBuilder();
                while ((line = bufferedReader.readLine()) != null) {
                    sb.append(line);
                }
                bufferedReader.close();
                Toast.makeText(this, sb.toString(), Toast.LENGTH_SHORT).show();
            } catch (IOException e) {
                throw new RuntimeException(e);
            }
        }
    }
}

```

SQLite

- Light-weight component, embedded relational database
- Built-in to Android
- Stores data in table using SQL
- Offline data storage
- Structured data (Rows & Columns)
- Fast performance
- Secure (App-private by default)



```
public class SQLiteHelper extends SQLiteOpenHelper {
    private static final String DATABASE_NAME = "students.db";
    private static final int DB_VERSION = 1;
    private static SQLiteHelper sqliteHelper;

    public static synchronized SQLiteHelper getInstance(Context context) {
        if (sqliteHelper == null) {
            sqliteHelper = new SQLiteHelper(context.getApplicationContext());
        }
        return sqliteHelper;
    }
}
```

```
private SQLiteHelper(@Nullable Context context) {
    super(context, SQLiteHelper.DATABASE_NAME, null, SQLiteHelper.DB_VERSION);
}
```

Private Constructor -> Prevent create new instance within outside class

Make Single Instance - Lazy initialization (Make sure one instance for the app)

```
@Override
public void onCreate(SQLiteDatabase sqLiteDatabase) {
    String studentTableQuery = "CREATE TABLE IF NOT EXISTS `student` " +
        "(id INTEGER PRIMARY KEY AUTOINCREMENT, " + // use AUTOINCREMENT instead of mysql AUTO_INCREMENT
        "name VARCHAR(150), " +
        "age INTEGER)";
    sqLiteDatabase.execSQL(studentTableQuery);
}
```

Database table creation part

```
@Override
public void onUpgrade(SQLiteDatabase sqLiteDatabase, int oldVersion, int newVersion) {
    String dropStudentTable = "DROP TABLE IF EXISTS `student`";
    sqLiteDatabase.execSQL(dropStudentTable);
    onCreate(sqLiteDatabase);
}
```

Trigger when database update
Use table modification

```
private void insertData() {
    SQLiteHelper sqliteHelper = SQLiteHelper.getInstance(this);
    SQLiteDatabase writableDatabase = sqliteHelper.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put("name", "Ramin Senanayake");
    values.put("age", "22");
    writableDatabase.insert("student", null, values);
}
```

```

private void updateData() {
    SQLiteHelper sqLiteHelper = SQLiteHelper.getInstance(this);
    SQLiteDatabase writableDatabase = sqLiteHelper.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put("age", "23");
    /// UPDATE `student` SET `age`= ? WHERE `id`= ?
    /// new String[]{1} -> Incorrect data passing (String array != Integer value)
    /// complete query ==> UPDATE `student` SET `age`=25 WHERE `id`=1;
    writableDatabase.update("student", values, "id=?", new String[]{String.valueOf(1)});
}

```

```

private void deleteData() {
    SQLiteHelper sqLiteHelper = SQLiteHelper.getInstance(this);
    SQLiteDatabase writableDatabase = sqLiteHelper.getWritableDatabase();
    int deleted = writableDatabase.delete("student", "id=?", new String[]
{String.valueOf(1)});
    Log.i(SQLiteHelper.class.getSimpleName(), "deleteData: " + deleted);
}

```

Network Connection

URLConnection

```

1 private static final int SUCCESS_CODE = 200;
2
3 private void requestUserData() {
4
5     try (ExecutorService executorService = Executors.newSingleThreadExecutor()) {
6         executorService.execute(new Runnable() {
7             @Override
8             public void run() {
9                 try {
10                     URL url = new
11 URL("https://jsonplaceholder.typicode.com/users/1");
12                     HttpURLConnection connection = (HttpURLConnection)
13 url.openConnection();
14                     connection.setRequestMethod("GET");
15                     connection.setConnectTimeout(10000);
16                     connection.setReadTimeout(10000);
17                     int responseCode = connection.getResponseCode();
18                     if (responseCode == SUCCESS_CODE) {
19                         BufferedReader bufferedReader = new BufferedReader(new
20 InputStreamReader(connection.getInputStream()));
21                         StringBuilder sb = new StringBuilder();
22                         String line;
23                         while ((line = bufferedReader.readLine()) != null) {
24                             sb.append(line);
25                         }
26                         Log.d(MainActivity.class.getSimpleName(), "responseCode: "
27 + sb.toString());
28                         JSONObject jsonObject = new JSONObject(sb.toString());
29                         String name = jsonObject.getString("name");
30                         Log.d(MainActivity.class.getSimpleName(), "Name: " + name);
31                     }
32                 } catch (IOException | JSONException e) {
33                     throw new RuntimeException(e);
34                 }
35             }
36         });
37     }
38 }

```

Retrofit

- Retrofit is a type-safe HTTP client



Retrofit Client - single instance

```
public class RetrofitClient {
    private static Retrofit retrofit;
    public static final String BASE_URL = "http://192.168.8.208:8080/api/v1/";
    /// emulators => [...] (http://10.0.2.2:8080/api/v1)

    public static Retrofit getInstance(Context context) {
        if (retrofit == null) {
            OkHttpClient client = new OkHttpClient.Builder()
                .addInterceptor(new AuthInterceptor(context))
                .build();

            retrofit = new Retrofit.Builder()
                .baseUrl(BASE_URL)
                .client(client)
                .addConverterFactory(GsonConverterFactory.create())
                .build();
        }
        return retrofit;
    }
}
```

Network Connection

Retrofit Network

Retrofit Object

- Set BASE URL
- Attach Gson convertor
- Set OkHttpClient (Interceptor)

```
new Retrofit.Builder()
    .baseUrl("http://192.168.4.1:8080/api/v1")
    .client(client)
    .addConverterFactory(GsonConverterFactory.create())
    .build();
```

API Interface (APIService)

- Define server end points
- Describe HTTP requests
- Actual code generated by Retrofit (runtime proxy)
- @GET, @POST, @PUT, @DELETE

```
public interface AuthApi {
    @POST("auth/refresh")
    Call<TokenDTO> refreshAccessToken(@Body TokenDTO tokenDTO);

    @POST("auth/login")
    Call<TokenDTO> userLogin(@Body LoginRequestDTO requestDTO);
}
```

Call Object

- Represent a single API request
- enqueue() → asynchronous
- execute() → synchronized (**Don't use UI Thread**)

```
Call<TokenDTO> refreshAccessToken(@Body TokenDTO tokenDTO);
```

Callback Object

- Handle responses (Success, Network Errors)

```
call.enqueue(new Callback<List<StudentDTO>>() {
    @Override
    public void onResponse(Call<List<StudentDTO>> call,
        Response<List<StudentDTO>> response) {
        ...
    }
})

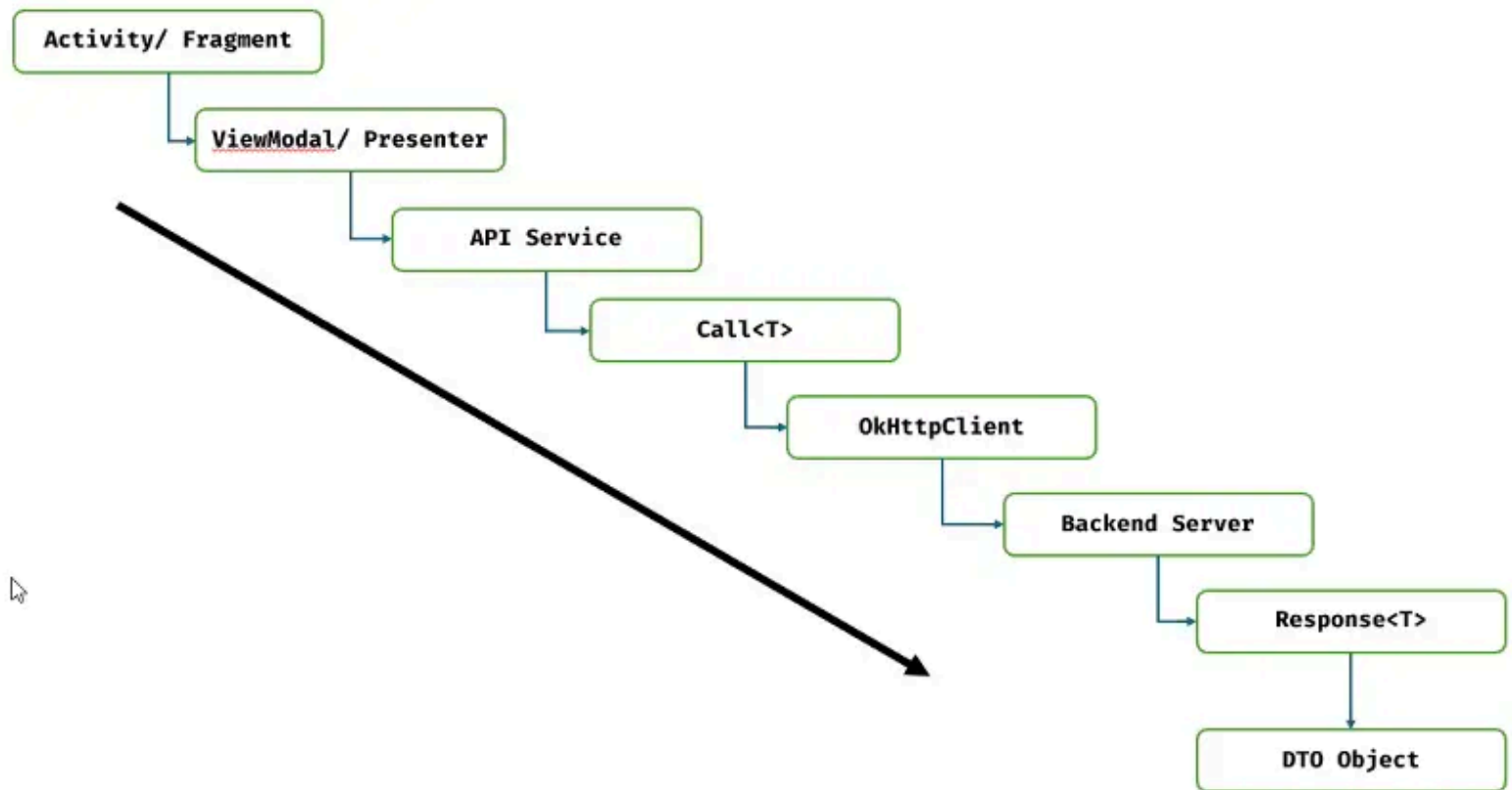
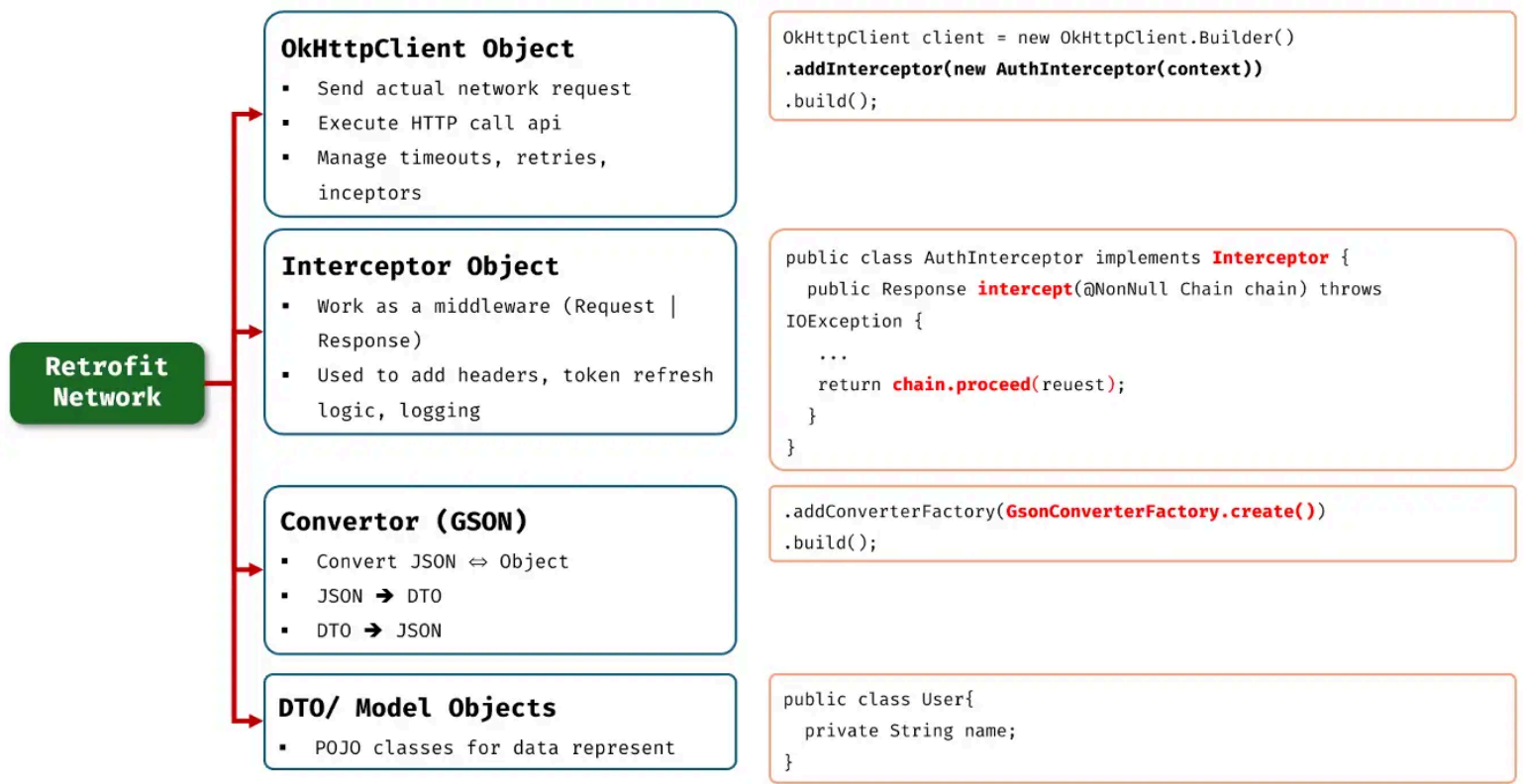
@Override
public void onFailure(Call<List<StudentDTO>> call, Throwable t) {
    ...
}
```

Retrofit Network

Response Object

- Represent server response
- response.body() → actual data
- response.code() → HTTP status code
- response.isSuccessful() → Check status code between 200-299

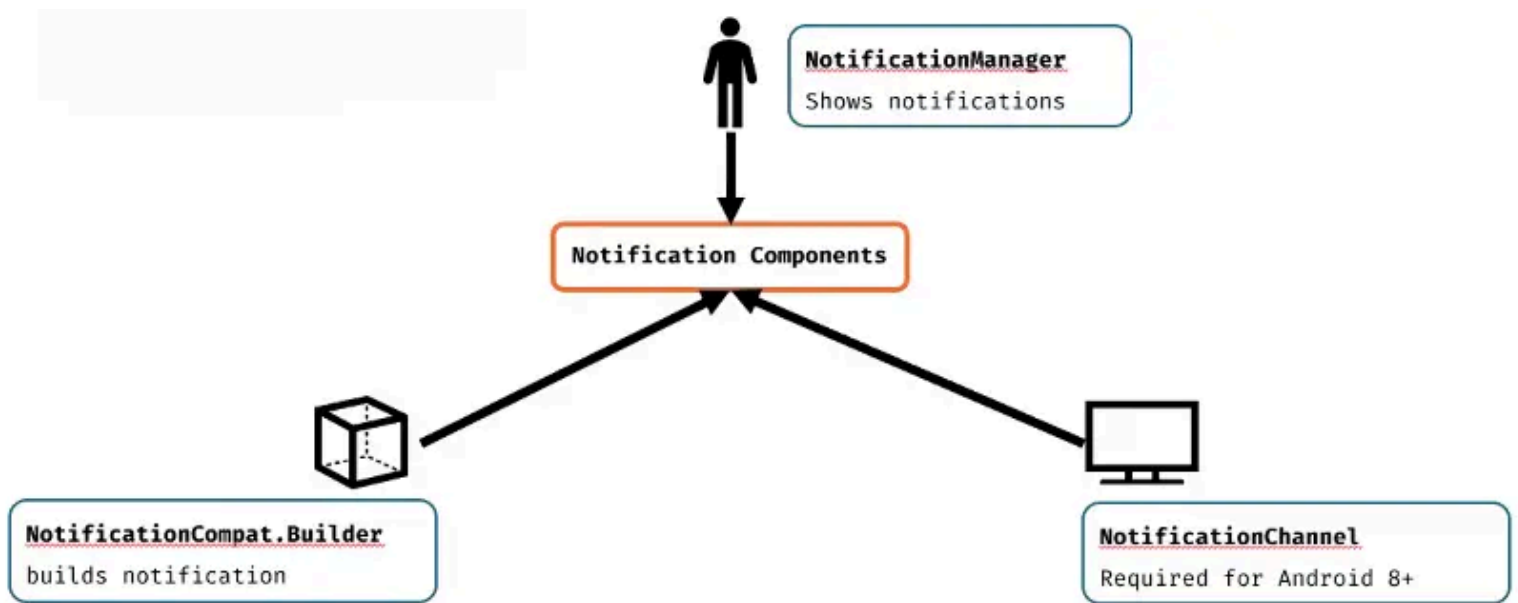
```
Response<User> response
```



Notification

- Notification එකක් කියලා කියන්නේ app එක ඔපන් කරලා නැති වෙලාවකත් message එකක් පෙන්වන්න පාවිච්චි කරන Android feature එකක්
- Key Features

- Appear on status bar
- Works when app closed
- Can contain action (open app)
- Managed by NotificationManager



Notification Sending Process



AndroidManifest.xml

```
<manifest>
  <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
  <application ...> ... </application>
</manifest>
```



MainActivity.java

```

private static final String CHANNEL_ID = "my_channel_01";

private final ActivityResultLauncher<String> requestPermissionLauncher =
registerForActivityResult(
    /// when trigger permission allowed or not
    new ActivityResultContracts.RequestPermission(), isGranted -> {
        if (isGranted) {
            Toast.makeText(this, "Notification permission granted",
Toast.LENGTH_SHORT).show();
        } else {
            Toast.makeText(this, "Permission not allowed!",
Toast.LENGTH_SHORT).show();
        }
    }
);

/// Check runtime permission
private void checkAndRequestPermission() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) { ///
Android version >= Android version 13
        if (ContextCompat.checkSelfPermission(this,
Manifest.permission.POST_NOTIFICATIONS) != PackageManager.PERMISSION_GRANTED) {

requestPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS);
        }
    }
}

private void createNotificationChannel() {
    /// Our android OS version >= Android version 8 (API level 26)
    /// minSdk = 28 always true
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        CharSequence name = "General Channel";
        String description = "Channel for general notifications";
        int importance = NotificationManager.IMPORTANCE_HIGH;
        NotificationChannel channel = new NotificationChannel(CHANNEL_ID, name,
importance);
        channel.setDescription(description);
        NotificationManager notificationManager =
getSystemService(NotificationManager.class);
        notificationManager.createNotificationChannel(channel);
    }
}

```

```

    }
}

private void sendNotification() {
    Notification.Builder builder = new Notification.Builder(this,
CHANNEL_ID)
        .setSmallIcon(R.drawable.notifications)
        .setPriority(Notification.PRIORITY_HIGH)
        .setAutoCancel(true);
    NotificationManagerCompat notificationManager =
NotificationManagerCompat.from(this);
    if (ContextCompat.checkSelfPermission(this,
Manifest.permission.POST_NOTIFICATIONS) == PackageManager.PERMISSION_GRANTED) {
        int notificationId = 1;
        notificationManager.notify(notificationId, builder.build());
    } else {
        checkAndRequestPermission();
    }
}
}

```

```

Button notificationBtn = findViewById(R.id.notificationBtn);
notificationBtn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        ...
        sendNotification();
        ...
    }
});

```



```
private static final String CHANNEL_ID = "my_channel_01";
public static final String KEY_TEXT_REPLY = "key_text_reply";

private void sendNotification() {
    Intent intent = new Intent(this, NotificationActivity.class);
    int requestCode = 0;
    PendingIntent pendingIntent = PendingIntent.getActivity(this,
        requestCode, intent, PendingIntent.FLAG_MUTABLE);
    RemoteInput remoteInput = new RemoteInput.Builder(KEY_TEXT_REPLY) //
androidx.core.app
        .setLabel("Type a message...")
        .build();
    NotificationCompat.Action replyAction = new
NotificationCompat.Action.Builder(R.drawable.ic_launcher_foreground,
        "Reply",
        pendingIntent).addRemoteInput(remoteInput).build();
    NotificationCompat.Builder builder = new
NotificationCompat.Builder(this, CHANNEL_ID)
        .setSmallIcon(R.drawable.ic_launcher_foreground)
        .setContentTitle("New Message")
        .setContentText("Hello Ramin")
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .addAction(replyAction)
        .setAutoCancel(true);
    NotificationManagerCompat notificationManager =
NotificationManagerCompat.from(this);
    if (ContextCompat.checkSelfPermission(this,
Manifest.permission.POST_NOTIFICATIONS) == PackageManager.PERMISSION_GRANTED) {
        int notificationId = 1;
        notificationManager.notify(notificationId, builder.build());
    } else {
        checkAndRequestPermission();
    }
}
```



NotificationActivity.java

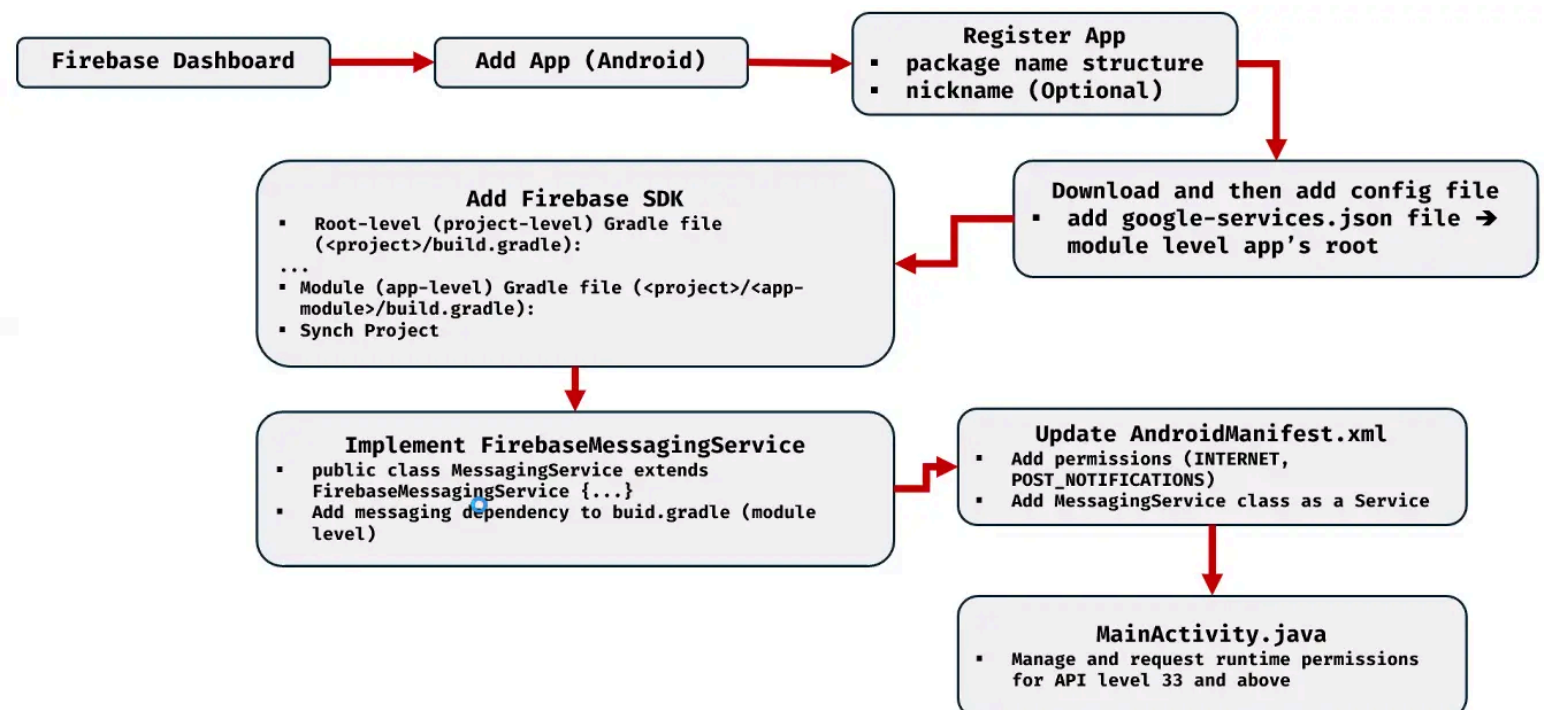
```
public class NotificationActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        ...

        Bundle bundle = RemoteInput.getResultsFromIntent(getIntent());
        if (bundle != null) {
            CharSequence charSequence =
                bundle.getCharSequence(MainActivity.KEY_TEXT_REPLY);
            Toast.makeText(this, charSequence, Toast.LENGTH_SHORT).show();
        }
    }
}
```

Push Notification

Add Firebase to Android App



```

package lk.raminsenanayake.pushnotifications.service;
...

import com.google.firebase.messaging.FirebaseMessagingService;
import com.google.firebase.messaging.RemoteMessage;
...

public class MessagingService extends FirebaseMessagingService {
    private static final String TAG = MessagingService.class.getSimpleName();

    @Override
    public void onNewToken(@NonNull String token) {
        Log.d(TAG, "Token: " + token);
    }

    @Override
    public void onMessageReceived(@NonNull RemoteMessage message) {
        String title = message.getNotification() != null ?
message.getNotification().getTitle() : "Title";
        String messageBody = message.getNotification() != null ?
message.getNotification().getBody() : "Empty Body";
        sendNotification(title, messageBody);
    }

    private void sendNotification(String title, String messageBody) {
        Intent intent = new Intent(this, MainActivity.class);
        intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);

        int requestCode = 0;
        PendingIntent pendingIntent = PendingIntent.getActivity(this, requestCode,
intent,
                PendingIntent.FLAG_ONE_SHOT | PendingIntent.FLAG_IMMUTABLE);

        String channelId = "default_channel";
        NotificationCompat.Builder notificationBuilder = new
NotificationCompat.Builder(this, channelId)
                .setSmallIcon(R.drawable.ic_launcher_foreground)
                .setContentTitle(title)
                .setContentText(messageBody)
                .setAutoCancel(true)
                .setContentIntent(pendingIntent);

```

```

        NotificationManager notificationManager = (NotificationManager)
getSystemService(NOTIFICATION_SERVICE);
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            NotificationChannel channel = new NotificationChannel(channelId, "Default
Channel", NotificationManager.IMPORTANCE_DEFAULT);
            notificationManager.createNotificationChannel(channel);
        }
        int notificationId = 1;
        notificationManager.notify(notificationId, notificationBuilder.build());
    }
}

```



AndroidManifest.xml

```

<manifest ...>

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

    <application ...>

        <service
            android:name=".service.MessagingService"
            android:exported="false">
            <intent-filter>
                <action android:name="com.google.firebase.MESSAGING_EVENT" />
            </intent-filter>
        </service>
        ...
    </application>

</manifest>

```



```
public class MainActivity extends AppCompatActivity {
    private final ActivityResultLauncher<String> requestPermissionLauncher =
        registerForActivityResult(new
ActivityResultContracts.RequestPermission(), isGranted -> {
        if (isGranted) {
            Toast.makeText(this, "Permission Granted",
Toast.LENGTH_SHORT).show();
        } else {
            Toast.makeText(this, "OOPS... Permission not allowed!",
Toast.LENGTH_SHORT).show();
        }
    });

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        ...
        checkAndRequestPermission();
    }

    /// For android 13 or higher (SDK 33^)
    private void checkAndRequestPermission() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            if (ContextCompat.checkSelfPermission(this,
Manifest.permission.POST_NOTIFICATIONS) != PackageManager.PERMISSION_GRANTED) {

requestPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS);
            }
        }
    }
}
```